Trabajo Práctico Integrador I y II

Asignatura: Arquitectura Web – FP-UNA 2026

Despliegue, conectividad, seguridad TLS 1.3, verificación experimental, resiliencia y observabilidad de una aplicación web

Autores: Sofia Mereles — Alba Martinez — Guillermo Gonzalez
Institución: Facultad Politécnica – Universidad Nacional de Asunción (FP-UNA)
Fecha: Noviembre 2026 – Avance hasta Fase 5 A3

Resumen

El presente informe documenta las Fases 1, 2, 3 y 4 del proyecto integrador de la asignatura **Arquitectura Web**. En la primera etapa se estableció la línea base de la infraestructura servidor mediante la configuración de un entorno virtualizado en Ubuntu Server en modo puente (*Bridged*), evaluando y seleccionando la plataforma **BookStack**. En la segunda etapa se llevó a cabo el despliegue del stack LNMP (Linux, Nginx, MariaDB, PHP 8.5-FPM), la ejecución de migraciones de base de datos y la verificación del servicio HTTP. En la tercera etapa se implementó la capa de seguridad con TLS 1.3 sobre el dominio `bookstack.local`, la redirección automática HTTPS, la evaluación experimental del rendimiento de la negociación criptográfica, la inspección de cabeceras HTTP/Gzip/Caché, la comparativa de arquitectura entre proxy e interfaz FastCGI, y el análisis de trazas de red mediante `tcpdump`. Finalmente, en la cuarta etapa se realizó el endurecimiento (*hardening*) del firewall UFW, la auditoría avanzada de cabeceras de seguridad, la simulación de fallos y resiliencia de servicios, pruebas de carga concurrente con `ab`, la observabilidad en tiempo real con `htop` y la matriz de pruebas consolidada.

Continuación del TP II. Se incorpora el análisis A1 de la Fase 5, con inventario del esquema real, matriz de acoplamiento respaldada por código y mapas de contextos del monolito y de su integración. El análisis documenta los seis módulos funcionales y las evidencias F5-E1 a F5-E3. Se incorporan también los ADR-006 a ADR-010 del apartado A2, con decisiones, alternativas y consecuencias para la evolución de la plataforma. El apartado A3 define los contratos HTTP y de eventos, su política de versionado y la validación reproducible F5-E4.

Índice

1. Objetivos, Alcance y Criterios de Aceptación **4**

2. Selección de la Aplicación Web **4**
2.1. Evaluación de Candidatos 4
2.2. Aplicación Seleccionada: BookStack 4

3. Arquitectura del Sistema ("As Built") **5**
3.1. Registro de Decisiones de Arquitectura (ADR) 6

4. Infraestructura y Configuración Base (Fase 1) **7**
4.1. Repositorio del Proyecto y Control de Versiones 7
4.2. Especificaciones de la Máquina Virtual 8
4.3. Línea Base de Red y Direccionamiento 9

5. Despliegue y Configuración de Servicios (Fase 2) **12**
5.1. Estado Inicial de los Servicios (F2-E1) 12
5.2. Verificación de Versiones de Runtime (F2-E2) 12
5.3. Creación y Configuración de la Base de Datos (F2-E3) 13
5.4. Ejecución de Migraciones de BookStack / Laravel (F2-E4) 14
5.5. Configuración del Virtual Host y Sintaxis de Nginx (F2-E5) 16
5.6. Estado Final de Servicios y Puertos en Escucha (F2-E6) 16
5.7. Pruebas de Conectividad HTTP y Verificación Web Final (F2-E7) 17

6. Seguridad, Rendimiento y Análisis de Red (Fase 3) **19**
6.1. Resolución de Nombres Local y Certificado X.509 19
6.2. Configuración de la Aplicación y Server Block en Nginx 20
6.3. Evaluación Experimental de Rendimiento HTTPS 21
6.4. Inspección de Cabeceras HTTP, Compresión y Caché 22
6.5. Comparativa de Arquitectura: Acceso Directo al Backend vs. Proxy Inverso 23
6.6. Captura y Análisis de Tráfico de Red con `tcpdump` 24
6.7. Verificación Final desde Navegador con DevTools (F3-E8) 25

7. Resiliencia, Seguridad Avanzada, Cierre y Defensa (Fase 4) **27**
7.1. Verificación de Firewall y Superficie Expuesta 27
7.2. Auditoría y Fortalecimiento de Cabeceras de Seguridad 28
7.3. Pruebas Funcionales, Manejo de Errores y Tolerancia a Fallos 29
7.4. Pruebas de Concurrencia y Monitoreo de Observabilidad 31
7.5. Registro de Corrección de Hallazgos y Gestión de Riesgos 33

8. Matriz de Pruebas y Evidencias Consolidada **34**

9. Conclusión del TP Integrador I **35**

10. Fase 5: Diseño y Descomposición del Monolito **37**
10.1. A1: Análisis del Monolito y Mapa de Contextos 37
10.1.1. Método y procedencia de la evidencia 37

10.1.2. F5-E1: Inventario del esquema y módulos funcionales 38
10.1.3. F5-E2: Matriz módulo por tabla 43
10.1.4. Trazabilidad de los accesos y dependencias compartidas 44
10.1.5. Efecto de los acoplamientos sobre la extracción 50
10.1.6. F5-E3: Mapa de contextos 51
10.1.7. Matriz de pruebas de A1 y resultado 53
10.2. A2: Registro de Decisiones de Arquitectura 54
10.2.1. ADR-006 — Estrategia de migración e integración con el monolito 54
10.2.2. ADR-007 — Selección del broker de mensajes 56
10.2.3. ADR-008 — Plataforma de orquestación 59
10.2.4. ADR-009 — Base de datos por servicio 62
10.2.5. ADR-010 — Consistencia eventual, idempotencia y mensajes fallidos 65
10.2.6. Resultado de A2 68
10.3. A3: Contratos de API y de Eventos 69
10.3.1. Contratos HTTP en OpenAPI 3.1 69
10.3.2. AsyncAPI 3.0 y sobre de evento común 70
10.3.3. Adaptación del Webhook y aceptación durable 71
10.3.4. Política de versionado y compatibilidad 72
10.3.5. F5-E4: Validación de especificaciones y ejemplos 72

1. Objetivos, Alcance y Criterios de Aceptación

El objetivo general del proyecto es diseñar, desplegar y auditar una infraestructura servidor segura, resiliente y observable, ejecutando sobre ella la aplicación web de gestión del conocimiento **BookStack**.

- **Alcance (Fase 1):** Preparación de la máquina virtual (2 vCPU, 4 GB RAM, 20 GB Disco), asignación de direccionamiento IP en capa 3, habilitación de OpenSSH, verificación de conectividad ICMP bidireccional y definición de la arquitectura lógica de tres capas.
- **Alcance (Fase 2):** Despliegue del stack LNMP; creación del esquema relacional en MariaDB; ejecución de migraciones de Laravel/BookStack; configuración del servidor virtual en Nginx y validación del flujo HTTP.
- **Alcance (Fase 3):** Configuración de resolución de nombres local (`bookstack.local`); generación e inspección de certificado digital X.509 con SAN; habilitación de HTTPS (TLS 1.3) con redirección 301; evaluación experimental de latencia con `curl`; auditoría de compresión Gzip y caché (304 Not Modified); comparativa técnica de arquitectura y análisis de traza de red con `tcpdump`.
- **Alcance (Fase 4):** Restricción de superficie de ataque mediante firewall UFW; auditoría de cabeceras de seguridad HTTP y atributos de cookies (`Secure`, `HttpOnly`, `SameSite`); validación de resiliencia ante caída del backend (HTTP 502) y autoarranque tras reinicio; pruebas de carga concurrente con `ab`; monitoreo de recursos en vivo (`htop`) y consolidación de evidencias.
- **Criterios de Aceptación:** Comunicación ICMP sin pérdida de paquetes, protocolo TLS 1.3 activo en puerto 443/TCP, redirección permanente HTTP 301 desde el puerto 80/TCP, cookies de sesión marcadas como `Secure` e `HttpOnly`, 0 % de fallos en pruebas de carga concurrente de 1.000 peticiones, y autoarranque verificado de todos los demonios del sistema tras un reinicio físico de la VM.

2. Selección de la Aplicación Web

Para la selección de la plataforma web se evaluaron dos candidatos de código abierto representativos de diferentes paradigmas arquitectónicos: **BookStack** (PHP 8.x / Laravel) y **Ghost** (Node.js / Express).

2.1. Evaluación de Candidatos

2.2. Aplicación Seleccionada: BookStack

Tras evaluar la mantenibilidad, el consumo de memoria y la compatibilidad con el entorno Linux empresarial, se seleccionó **BookStack**.

Criterio	Candidato 1: BookStack	Candidato 2: Ghost
Repositorio	https://github.com/BookStackApp/BookStack	https://github.com/TryGhost/Ghost
Licencia	MIT License	MIT License
Stack Tecnológico	PHP 8.x / Laravel Framework	Node.js / Express.js
Base de Datos	MariaDB 11.8+ / MySQL 8.0	MySQL 8.0
Servidor Web	Nginx + PHP-FPM (Unix Socket)	Nginx + Node.js Process Manager
Viabilidad	**Alta** (Instalación nativa predecible)	**Media** (Dependencias estrictas de Node)

Tabla 1: Comparativa técnica de candidatos para la selección de la aplicación web.

- **Repositorio Oficial:** https://github.com/BookStackApp/BookStack
- **Release y Commit Hash:** Release v26.05.4 (Commit `cec78b1`)
- **Arquitectura interna:** PHP 8.2+ / Laravel Framework
- **Servidor de Aplicación:** PHP-FPM vinculado vía Unix Domain Socket a Nginx
- **Base de Datos:** MariaDB 11.8+ listening en `127.0.0.1:3306`
- **Justificación:** Presenta una clara arquitectura de 3 capas, bajo consumo de memoria RAM (< 500 MB en régimen normal), facilidad de auditoría de logs y excelente integración con el servidor web Nginx.

3. Arquitectura del Sistema ("As Built")

La Figura 1 ilustra la topología lógica de tres capas implementada en la máquina virtual, detallando el flujo de protocolos, puertos y aislamiento de componentes.

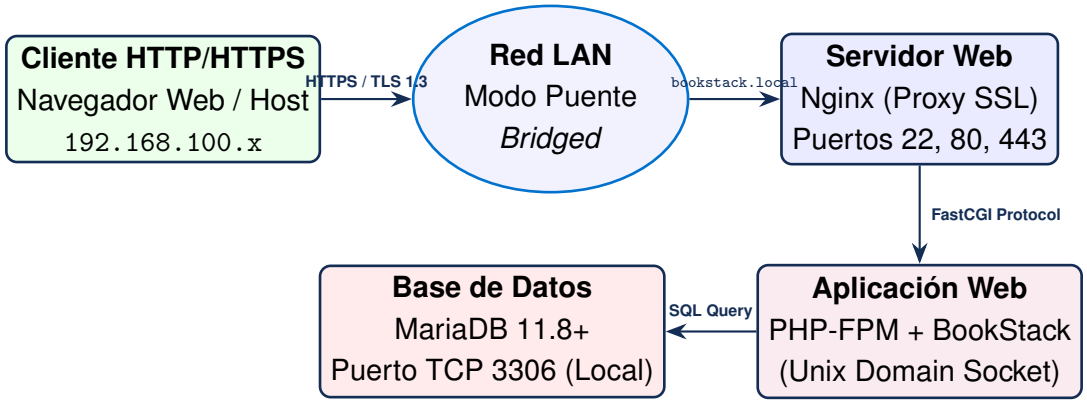


Figura 1: Diagrama de arquitectura "As Built" de 3 capas con terminación TLS y Hardening de Firewall.

3.1. Registro de Decisiones de Arquitectura (ADR)

ADR-001: Selección de la Aplicación Web de Código Abierto

Estatus: Aprobado.
Contexto: Se requiere desplegar una aplicación web orientada a la gestión documental en Ubuntu Server, garantizando observabilidad y bajo consumo de recursos.
Decisión: Seleccionar BookStack (PHP 8.5 / Laravel / MariaDB) sobre Ghost (Node.js).
Consecuencias: Arquitectura clásica de tres capas, predecible y altamente compatible con Nginx y PHP-FPM.

ADR-002: Modo de Red del Hipervisor (VMware Workstation)

Estatus: Aprobado.
Contexto: La VM requiere salida a Internet para instalar paquetes y accesibilidad directa desde la máquina anfitriona.
Decisión: Configurar el adaptador de red virtual en Modo Puente (*Bridged*).
Consecuencias: La VM obtiene una dirección IP de la red LAN física (`192.168.100.23/24`), facilitando la prueba de conectividad bidireccional.

ADR-003: Servidor Web y Procesador de Aplicación

Estatus: Aprobado.
Contexto: Se requiere un servidor web para gestionar peticiones HTTP/HTTPS de alta concurrencia y procesar código dinámico PHP.
Decisión: Utilizar Nginx como proxy inverso coordinado con PHP 8.5-FPM mediante un Unix Domain Socket (`/var/run/php/php8.5-fpm.sock`).
Consecuencias: Eficiencia en el manejo de conexiones concurrentes y entrega ultrarrápida de archivos estáticos.

ADR-004: Implementación de Terminación TLS 1.3 y Redirección HTTPS

Estatus: Aprobado.
Contexto: Asegurar la confidencialidad de las credenciales y el tráfico web.
Decisión: Implementar terminación TLS en Nginx con un certificado X.509 autofirmado para `bookstack.local`, forzando la redirección HTTP 301 desde el puerto 80 al 443.
Consecuencias: Cifrado end-to-end hacia el cliente; el backend PHP-FPM atiende únicamente peticiones procesadas y sanitizadas por Nginx.

ADR-005: Política Defensiva de Firewall (UFW)

Estatus: Aprobado.
Contexto: Reducir la superficie de ataque bloqueando puertos innecesarios.
Decisión: Aplicar política predeterminada `default deny incoming`, autorizando únicamente los puertos TCP 22, 80 y 443.
Consecuencias: El puerto 3306 de MariaDB queda aislado exclusivamente para peticiones locales de la interfaz de bucle invertido (`127.0.0.1`).

4. Infraestructura y Configuración Base (Fase 1)

4.1. Repositorio del Proyecto y Control de Versiones

Se estableció la línea base del proyecto creando el repositorio público en GitHub y vinculándolo con el entorno local de trabajo.



Figura 2: F1-E1: Comandos ejecutados en la terminal para la vinculación del repositorio local.

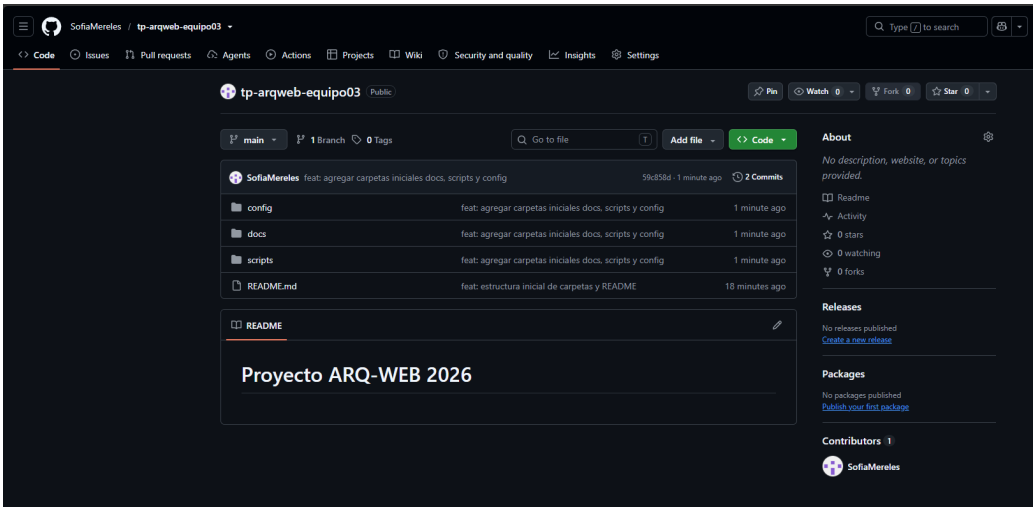


Figura 3: F1-E2: Vista web del repositorio público en GitHub con la estructura de directorios.

4.2. Especificaciones de la Máquina Virtual

- **Sistema Operativo:** Ubuntu Server 26.04 LTS (Kernel Linux 6.8.0 / 7.0.0-generic)
- **Recursos de Hardware:** 2 vCPU, 4096 MB RAM, 20 GB Disco SSD.
- **Adaptador de Red:** Virtual Network Adapter en Modo Puente (*Bridged*).

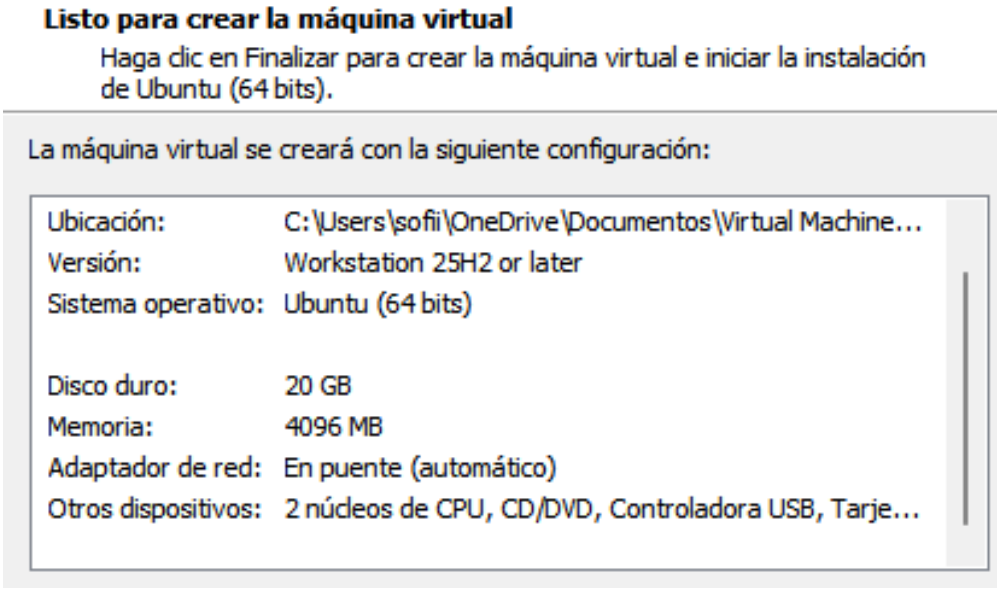


Figura 4: F1-E3: Configuración de hardware y red en VMware Workstation.



Figura 5: F1-E4: Consola de Ubuntu Server operativa con el usuario no-root iniciado.

4.3. Línea Base de Red y Direccionamiento

Mediante el adaptador en modo puente, la máquina virtual obtuvo la dirección IP `192.168.100.23/24` asignada por el servidor DHCP de la red LAN local.

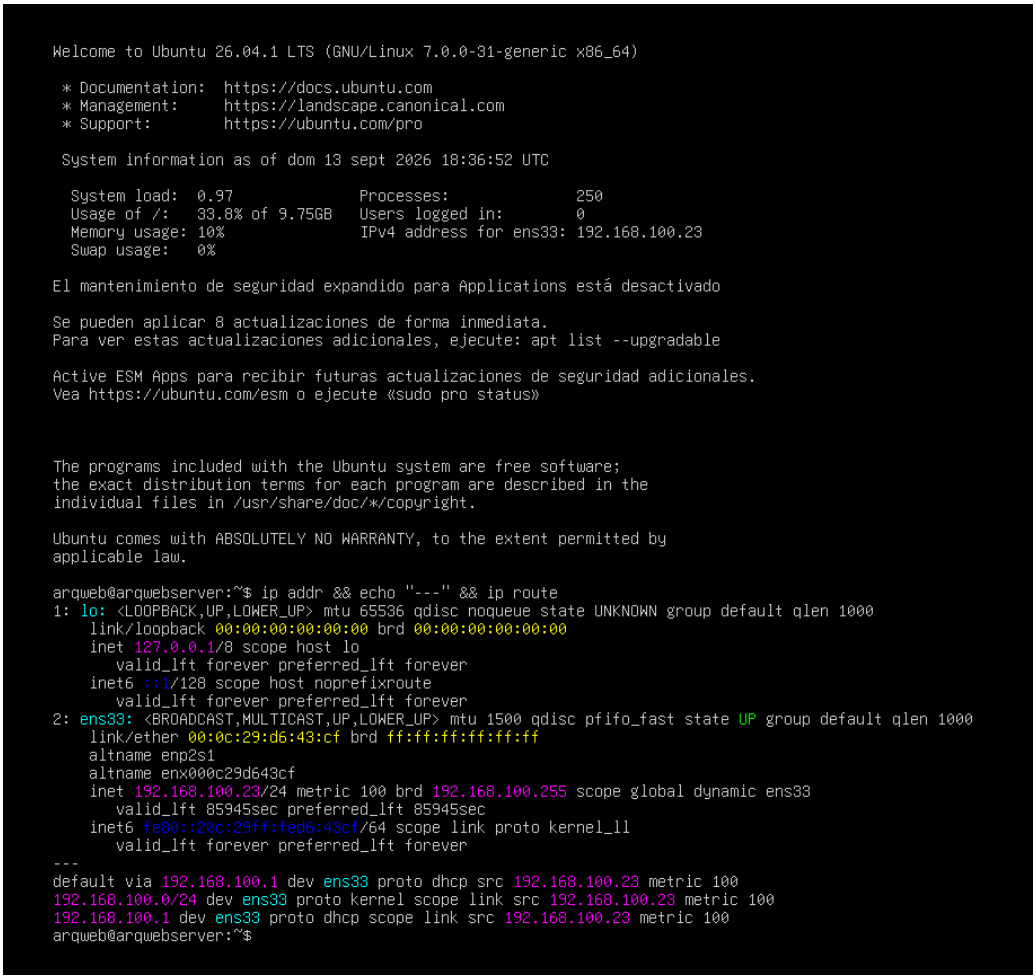


Figura 6: F1-E5: Salida del comando `ip addr` e `ip route` en Ubuntu Server.

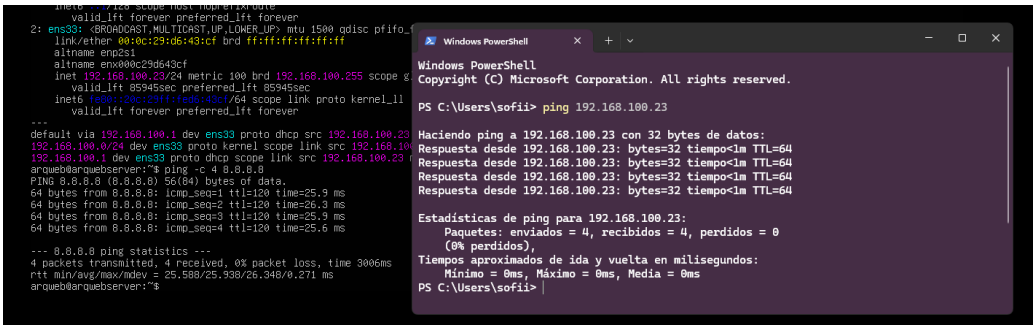


Figura 7: F1-E6: Pruebas de conectividad ICMP bidireccional y salida a Internet.

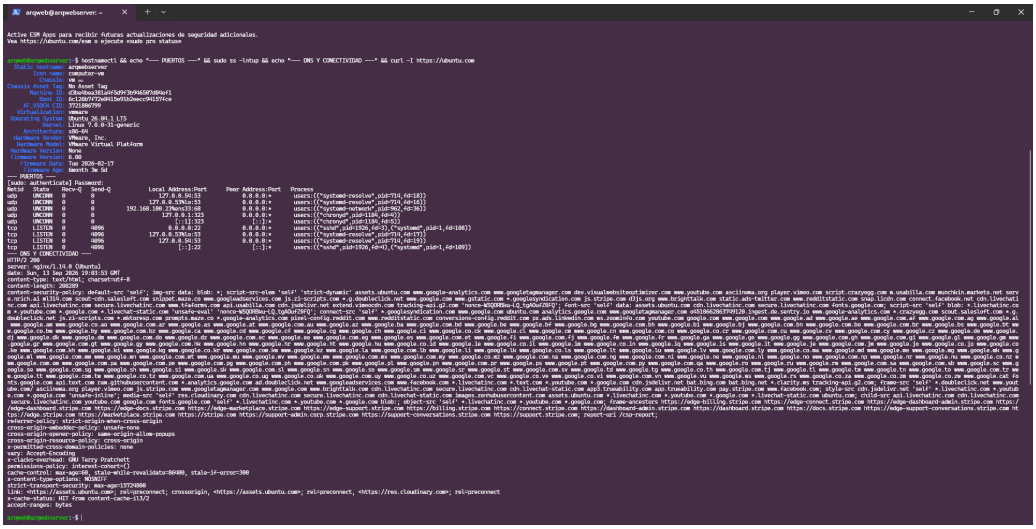


Figura 8: F1-E7: Verificación del estado del sistema, puerto 22 SSH y resolución DNS.

5. Despliegue y Configuración de Servicios (Fase 2)

En esta fase se llevó a cabo el despliegue del stack LNMP (Linux, Nginx, MariaDB, PHP 8.5-FPM) y la instalación completa de BookStack. A continuación se detalla técnicamente el procedimiento ejecutado en cada etapa.

5.1. Estado Inicial de los Servicios (F2-E1)

Antes de iniciar la instalación de los componentes web, se realizó una auditoría del estado inicial de los puertos y servicios en escucha mediante el comando `ss -tulpn >puerto_antes.txt`. Como se observa en la Figura 9, únicamente se encontraban activos el servicio OpenSSH en el puerto TCP 22 y el resolvedor de nombres local en el puerto UDP 53. Al intentar realizar una conexión HTTP local mediante `curl -I http://192.168.100.23`, el sistema retornó un rechazo explícito de conexión (*Connection refused*), confirmando que no existía ningún servidor web activo previo al despliegue.

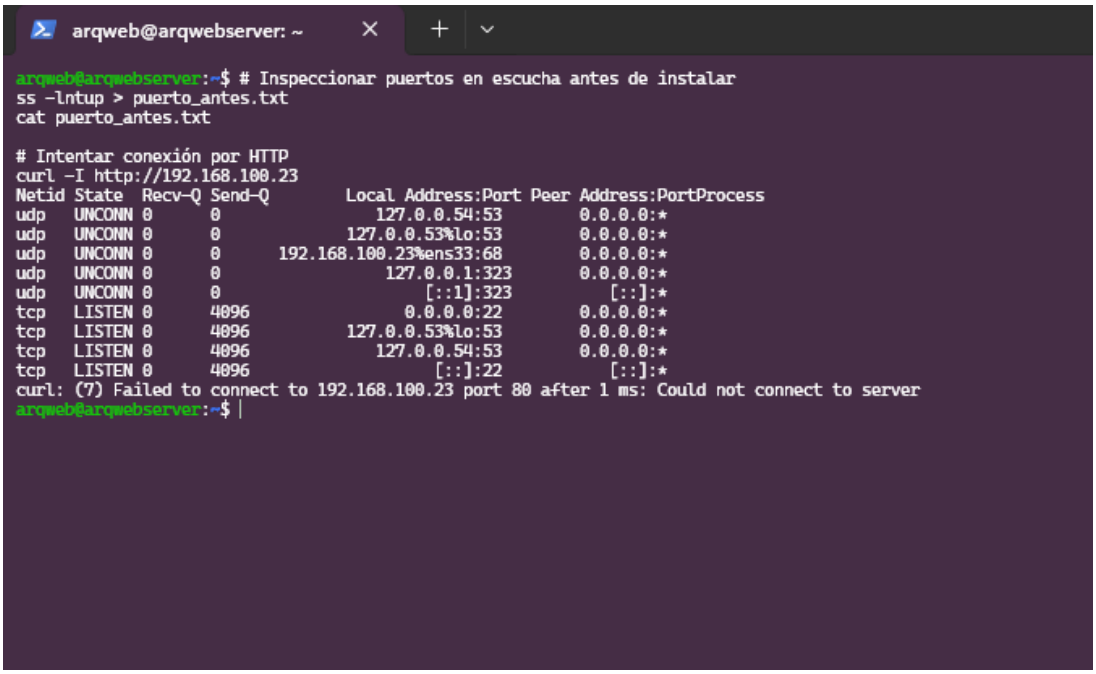


Figura 9: F2-E1: Verificación del estado inicial de los puertos antes de la instalación del stack.

5.2. Verificación de Versiones de Runtime (F2-E2)

Se procedió a la instalación de los paquetes del stack LNMP utilizando el gestor `apt`. Posterior a la instalación, se ejecutó una verificación unificada de las versiones del servidor web, del intérprete PHP y del motor de base de datos. Como evidencia la Figura 10, el entorno quedó configurado con **Nginx 1.28.3**, **PHP 8.5.4-cli** (con Zend OPcache v8.5.4 activo) y **MariaDB 11.8.6**. Todos los demonios correspondientes fueron habilitados en el gestor de servicios `systemd` para garantizar su inicio automático.

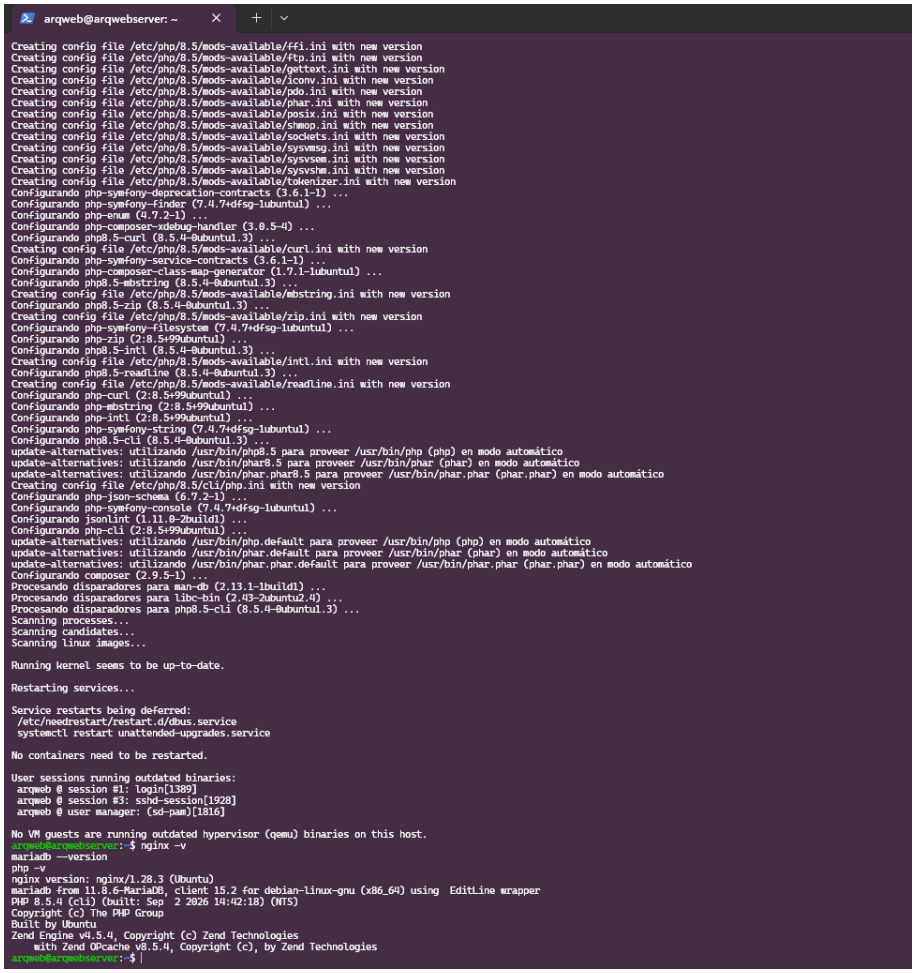


Figura 10: F2-E2: Comprobación de versiones activas de Nginx, PHP 8.5 y MariaDB.

5.3. Creación y Configuración de la Base de Datos (F2-E3)

Para dar soporte a la capa de persistencia de BookStack, se ingresó al gestor de base de datos MariaDB mediante la consola administrativa (`sudo mysql`). Se creó la base de datos relacional `bookstack_db` y se generó el usuario dedicado `bookstack_user` autenticado localmente (`@localhost`). Se otorgaron todos los privilegios sobre dicha base de datos y se aplicó la instrucción `FLUSH PRIVILEGES;`. La ejecución de `SHOW DATABASES;` confirmó la correcta creación de la estructura, tal como se aprecia en la Figura 11.

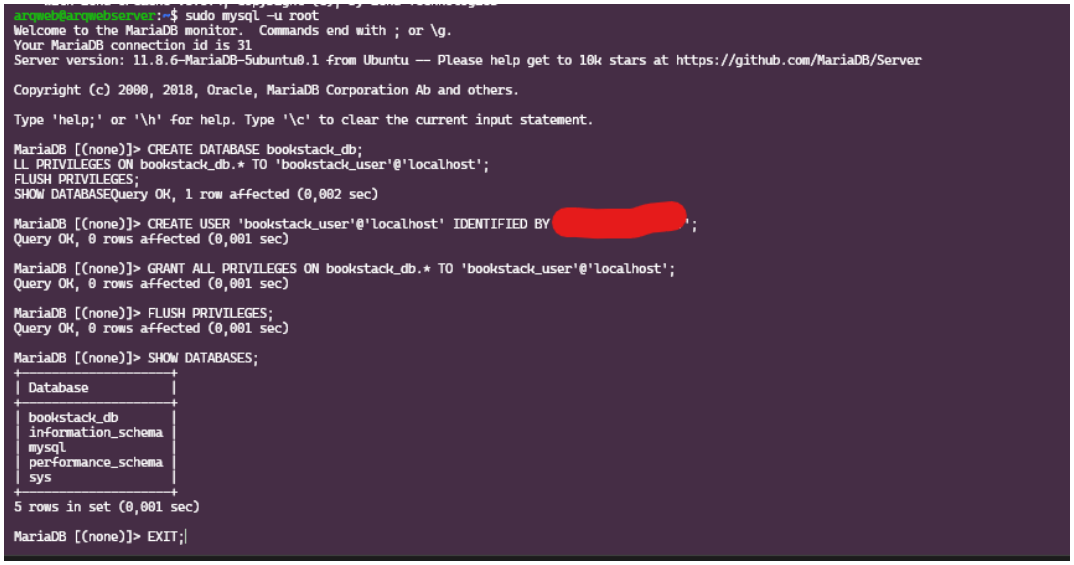


Figura 11: F2-E3: Sentencias SQL para la creación del esquema y asignación de permisos.

5.4. Ejecución de Migraciones de BookStack / Laravel (F2-E4)

Se desplegó el código fuente de BookStack Release v26.05.4 en la ruta `/var/www/bookstack`, asignando la propiedad de los archivos al usuario del servidor web (`www-data:www-data`) y otorgando permisos de escritura (`775`) sobre los directorios `storage` y `bootstrap/cache`. Se creó el archivo de entorno `.env` vinculando las credenciales de MariaDB y se generó la clave criptográfica de la aplicación mediante `php artisan key:generate --force`. Posteriormente, se ejecutó la migración del esquema utilizando `php artisan migrate --force`. Como demuestra la Figura 12, la herramienta Artisan ejecutó exitosamente más de 80 migraciones de tablas relacionales (`users`, `books`, `pages`, `roles`, `activities`, entre otras).



Figura 12: F2-E4: Ejecución exitosa de las migraciones de tablas en Laravel para BookStack.

5.5. Configuración del Virtual Host y Sintaxis de Nginx (F2-E5)

Se estructuró el archivo de Virtual Host en `/etc/nginx/sites-available/bookstack`, configurando la raíz web en `/var/www/bookstack/public` y derivando la ejecución de archivos PHP al socket Unix de FastCGI (`/var/run/php/php8.5-fpm.sock`). Se habilitó el sitio creando el enlace simbólico en `sites-enabled/` y se removió el sitio por defecto. Finalmente, se ejecutó `sudo nginx -t` para validar la sintaxis de la configuración. La Figura 13 certifica que el test resultó exitoso (*syntax is ok / test is successful*).

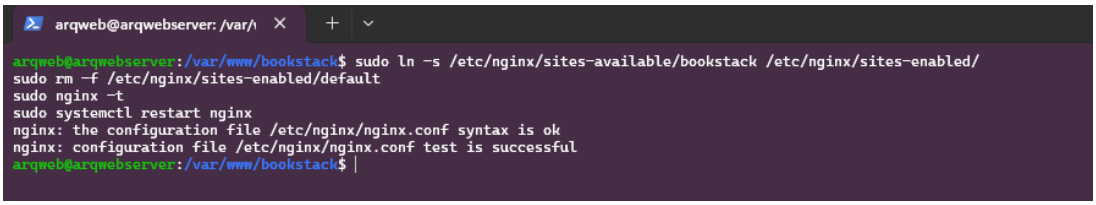


Figura 13: F2-E5: Verificación de sintaxis exitosa del archivo de configuración de Nginx.

5.6. Estado Final de Servicios y Puertos en Escucha (F2-E6)

Tras reiniciar el servidor web (`systemctl restart nginx`), se ejecutó una inspección detallada de sockets activos con `ss -tulpn`. Como ilustra la Figura 14, se constató que el proceso `nginx` se encuentra escuchando en el puerto TCP 80 (`0.0.0.0:80`) y el motor `mariadbd` se mantiene activo únicamente en la interfaz de bucle invertido TCP 3306 (`127.0.0.1:3306`), garantizando el aislamiento de la base de datos de la red pública.

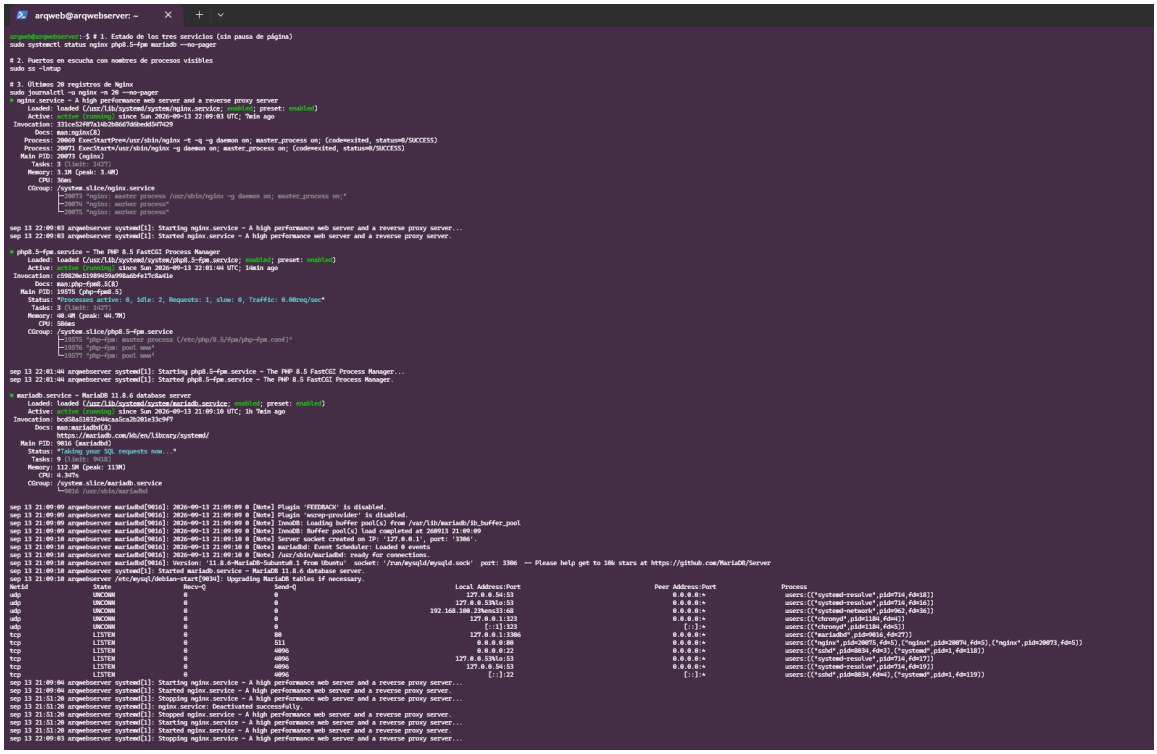


Figura 14: F2-E6: Inspección final de servicios activos y sockets en los puertos TCP 80 y TCP 3306.

5.7. Pruebas de Conectividad HTTP y Verificación Web Final (F2-E7)

Para finalizar la Fase 2, se realizó la prueba de conectividad HTTP a nivel de consola y navegador. En primer lugar, la ejecución de `curl -I http://192.168.100.23` retornó un código de respuesta `HTTP/1.1 200 OK` con las cabeceras emitidas por Nginx y la inicialización de la cookie de sesión de Laravel (Figura 15). En segundo lugar, se accedió a la URL desde la máquina anfitriona; la Figura 16 demuestra la renderización completa del formulario de inicio de sesión de BookStack y el panel DevTools de red registrando el estado 200 OK en la descarga de todos los recursos estáticos.

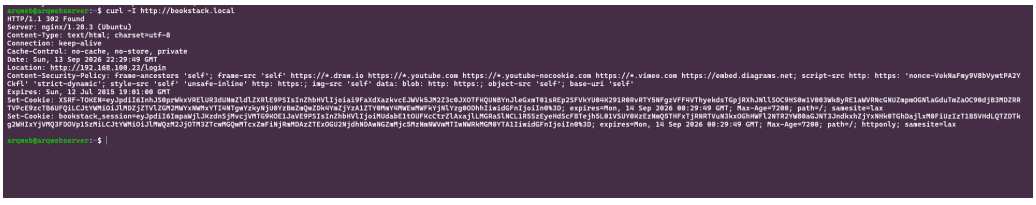


Figura 15: F2-E7a: Respuesta del servidor web mostrando encabezados HTTP mediante `curl`.

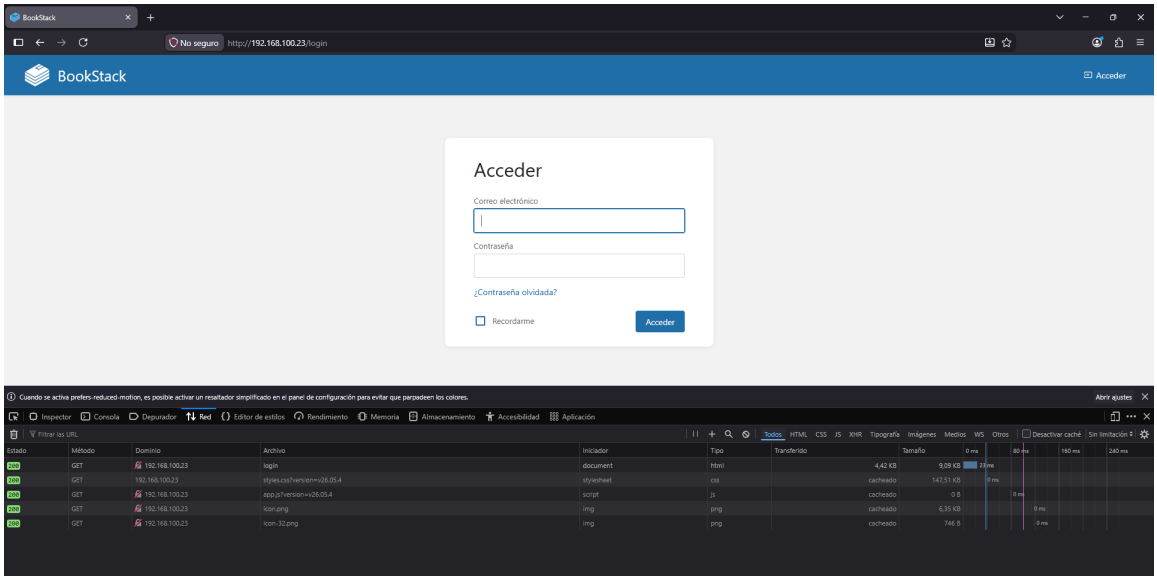


Figura 16: F2-E7b: Carga completa de la interfaz de login de BookStack y panel DevTools.

6. Seguridad, Rendimiento y Análisis de Red (Fase 3)

En esta sección se detalla la implementación del protocolo HTTPS sobre TLS 1.3, la resolución de nombres local, la optimización de cabeceras/caché, las mediciones de latencia y el análisis de la traza de red capturada.

6.1. Resolución de Nombres Local y Certificado X.509

Para simular un entorno de producción con un Fully Qualified Domain Name (FQDN), se asignó el dominio `bookstack.local` asociándolo a la dirección IP `192.168.100.23` dentro del archivo de mapeo estático del sistema (`/etc/hosts`).



Figura 17: F3-E1: Configuración de resolución de nombres en `/etc/hosts` y prueba con `ping bookstack.local`.

En la Figura 17 (Evidencia F3-E1), se comprueba que el nombre de dominio resuelve correctamente hacia la IP de la máquina virtual con un tiempo de latencia nulo. A continuación, se generó un certificado digital X.509 autofirmado mediante OpenSSL, especificando la extensión *Subject Alternative Name* (SAN):

```
arqweb@arqwebserver:~$ sudo ls -la /etc/ssl/certs/bookstack.crt /etc/ssl/private/bookstack.key
-rw-r--r-- 1 root root 1403 Sep 13 23:45 /etc/ssl/certs/bookstack.crt
-rw------- 1 root root 1704 Sep 13 23:45 /etc/ssl/private/bookstack.key
arqweb@arqwebserver:~$ # Sujeto y Emisor
sudo openssl x509 -in /etc/ssl/certs/bookstack.crt -noout -subject -issuer

# SAN (Subject Alternative Name)
sudo openssl x509 -in /etc/ssl/certs/bookstack.crt -noout -ext subjectAltName

# Fechas de vigencia (notBefore / notAfter)
sudo openssl x509 -in /etc/ssl/certs/bookstack.crt -noout -dates

# Huella digital SHA-256
sudo openssl x509 -in /etc/ssl/certs/bookstack.crt -noout -fingerprint -sha256
subject=C=PY, ST=Central, L=San Lorenzo, O=FP-UNA, OU=ArqWeb, CN=bookstack.local
issuer=C=PY, ST=Central, L=San Lorenzo, O=FP-UNA, OU=ArqWeb, CN=bookstack.local
X509v3 Subject Alternative Name:
    DNS:bookstack.local
notBefore=Sep 13 23:45:46 2026 GMT
notAfter=Sep 13 23:45:46 2027 GMT
sha256 Fingerprint=8D:8C:12:06:11:FD:1A:55:50:91:61:6D:F9:02:C6:00:34:1F:B7:7C:C7:97:9A:18:BF:BA:2B:23:2E:52:3F:2E
arqweb@arqwebserver:~$ s
```

Figura 18: F3-E2: Inspección del certificado X.509 emitido con OpenSSL (`CN=bookstack.local`, RSA 2048 bits).

Como evidencia la Figura 18 (Evidencia F3-E2), la inspección del archivo `/etc/ssl/certs/bookst` confirma la validez del certificado, vigencia de un año, algoritmo `sha256WithRSAEncryption` y huella digital correspondiente.

Análisis de la Cadena de Confianza y Advertencias del Navegador: Al acceder a un sitio configurado con un certificado autofirmado, el navegador emite una advertencia de seguridad (*"La conexión no es privada"*). Esto ocurre porque la Autoridad Certificadora (CA) emisora es la misma entidad que el sujeto (`Issuer = Subject`), por lo que la CA no se encuentra dentro del almacén de confianza del sistema operativo cliente (*Root CA Store*). En producción, este paso se sustituye por la emisión de un certificado validado por una CA pública (ej. Let's Encrypt).

6.2. Configuración de la Aplicación y Server Block en Nginx

Para garantizar la coherencia de la aplicación Laravel con la capa de transporte segura, se actualizó el archivo de entorno global `.env` de BookStack ajustando la variable de URL base:

```
arqweb@arqwebserver:~$ sudo nano /var/www/bookstack/.env
[sudo: authenticate] Password:
arqweb@arqwebserver:~$ grep APP_URL /var/www/bookstack/.env
APP_URL=https://bookstack.local
arqweb@arqwebserver:~$
```

Figura 19: F3-E3: Verificación de la variable de entorno `APP_URL=https://bookstack.local` en el archivo `.env`.

En la Figura 19 (Evidencia F3-E3), el comando `grep` confirma que los recursos estáticos, enlaces absolutos y cookies de sesión sean generados bajo el esquema seguro HTTPS. Posteriormente, se reestructuró el archivo de bloque de servidor de Nginx dividiéndolo en dos bloques `server`:

1. **Bloque HTTP (Puerto 80):** Captura todas las peticiones inseguras e inyecta una redirección permanente `HTTP 301 Moved Permanently` hacia `https://bookstack.local$reques`
2. **Bloque HTTPS (Puerto 443):** Configura la escucha en modo SSL (`listen 443 ssl;`), enlaza la clave privada y certificado, limita los protocolos a **TLS 1.2** y **TLS 1.3**, y deriva la ejecución PHP al socket Unix de PHP-FPM.

```
  GNU nano 8.7.1
server {
    listen 80;
    listen [::]:80;
    server_name bookstack.local;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;
    server_name bookstack.local;

    ssl_certificate /etc/ssl/certs/bookstack.crt;
    ssl_certificate_key /etc/ssl/private/bookstack.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    root /var/www/bookstack/public;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/var/run/php/php8.5-fpm.sock;
    }
}
```

Figura 20: F3-E4: Edición del Server Block en Nginx habilitando SSL/TLS 1.3 en el puerto 443 y redirección HTTP 301.

6.3. Evaluación Experimental de Rendimiento HTTPS

Se ejecutó un script en Bash realizando 10 iteraciones consecutivas mediante el comando `curl -w` para descomponer los tiempos de conexión en cada fase del modelo de red:

```
arqweb@arqwebserver:~$ for i in {1..10}; do
  echo -n "Medicion $i: "
  curl -k -s -w "DNS: %{time_namelookup}s | Connect: %{time_connect}s | TLS: %{time_appconnect}s | Total: %{time_total}s\n" -o /dev/null https://bookstack.local
done
Medicion 1: DNS: 0.000662s | Connect: 0.002223s | TLS: 0.012722s | Total: 0.038471s
Medicion 2: DNS: 0.000432s | Connect: 0.000560s | TLS: 0.012620s | Total: 0.028179s
Medicion 3: DNS: 0.000469s | Connect: 0.002177s | TLS: 0.007113s | Total: 0.022014s
Medicion 4: DNS: 0.000450s | Connect: 0.000638s | TLS: 0.005013s | Total: 0.019766s
Medicion 5: DNS: 0.000364s | Connect: 0.000483s | TLS: 0.005072s | Total: 0.021375s
Medicion 6: DNS: 0.000421s | Connect: 0.000543s | TLS: 0.005405s | Total: 0.021081s
Medicion 7: DNS: 0.000473s | Connect: 0.001106s | TLS: 0.005936s | Total: 0.021663s
Medicion 8: DNS: 0.000706s | Connect: 0.000891s | TLS: 0.005562s | Total: 0.021675s
Medicion 9: DNS: 0.000405s | Connect: 0.000504s | TLS: 0.004726s | Total: 0.021620s
Medicion 10: DNS: 0.000621s | Connect: 0.000959s | TLS: 0.005881s | Total: 0.021218s
arqweb@arqwebserver:~$
```

Figura 21: F3-E5: Ejecución secuencial de 10 mediciones de rendimiento con `curl` sobre HTTPS.

Como muestra la Figura 21 (Evidencia F3-E5), se registraron los métricos consolidados en la Tabla 2.

Interpretación Técnica de los Resultados:

Iteración	DNS Lookup (s)	TCP Connect (s)	TLS Handshake (s)	Tiempo Total (s)
1	0.000662	0.002223	0.012722	0.038471
2	0.000432	0.000560	0.012620	0.028179
3	0.000469	0.002177	0.007113	0.022014
4	0.000450	0.000638	0.005013	0.019766
5	0.000364	0.000483	0.005072	0.021375
6	0.000421	0.000543	0.005405	0.021081
7	0.000473	0.001106	0.005936	0.021663
8	0.000706	0.000891	0.005562	0.021675
9	0.000405	0.000504	0.004726	0.021620
10	0.000621	0.000959	0.005881	0.021218
Promedio	**0.000500s (0.50 ms)**	**0.001008s (1.01 ms)**	**0.007005s (7.00 ms)**	**0.023706s (23.71 ms)**

Tabla 2: Desglose de tiempos de respuesta en la negociación HTTPS (10 iteraciones).

- **Efecto Calentamiento (*Warm-up*):** La primera medición registró un tiempo total de $38{,}4$ ms debido a la lectura inicial en disco de los certificados y la inicialización de procesos en el worker de PHP-FPM.
- **Optimización por Reuso de Sesión TLS:** A partir de la iteración 4, el tiempo de negociación TLS disminuyó de $12{,}7$ ms a $\sim 5{,}0$ ms. Esto se debe al uso de *TLS Session Tickets* en TLS 1.3, eliminando un viaje de ida y vuelta (*RTT*) en reconexiones.
- **Estabilidad:** El sistema demostró un tiempo de procesamiento promedio de **23.71 ms**, manteniendo una desviación estándar mínima en el entorno virtualizado.

6.4. Inspección de Cabeceras HTTP, Compresión y Caché

Se analizó la respuesta del servidor mediante `curl -I` enviando la cabecera `Accept-Encoding: gzip` para auditar las políticas de caché y compresión de datos:



Figura 22: F3-E6: Verificación de cabeceras HTTP de respuesta: `Content-Encoding: gzip`, `Cache-Control` y cookies.

Como evidencia la Figura 22 (Evidencia F3-E6), se identifican los siguientes comportamientos:

- **Compresión Dinámica:** La presencia de `Content-Encoding: gzip` confirma que Nginx comprime el cuerpo HTML antes de la transmisión, reduciendo el consumo de ancho de banda.
- **Caché en Rutas Dinámicas:** La cabecera `Cache-Control: no-cache, no-store, private` previene que páginas transaccionales con tokens CSRF o cookies de sesión sean almacenadas en proxies intermedios.
- **Atributos de Cookies Seguras:** Las cookies `XSRF-TOKEN` y `bookstack_session` incluyen las directivas `secure` y `httponly`, garantizando que solo viajen sobre HTTPS y no sean accesibles mediante código JavaScript de cliente.
- **Validación Condicional 304 (Not Modified):** Para recursos estáticos, Nginx entrega la cabecera `Last-Modified`. Ante peticiones subsecuentes del cliente con `If-Modified-Since`, el servidor responde con `HTTP 304 Not Modified`, omitiendo el reenvío del archivo.

6.5. Comparativa de Arquitectura: Acceso Directo al Backend vs. Proxy Inverso

Conclusión de Arquitectura: No es seguro ni técnicamente viable exponer directamente PHP-FPM a la red. Nginx actúa como una capa indispensable de terminación SSL, optimización de caché, entrega de recursos estáticos y protección del motor de la aplicación.

Criterio	Acceso Directo (PHP-FPM)	Proxy Inverso (Nginx)
Protocolo Soportado	Exclusivamente protocolo binario **FastCGI**. No habla HTTP de forma nativa.	**HTTP/1.1, HTTP/2, HTTPS (TLS 1.2/1.3)**. Traduce HTTP a FastCGI.
Terminación TLS	Sin soporte nativo de cifrado X.509/TLS. Tráfico transmitido en texto plano.	Gestiona el certificado SSL/TLS, descifrado y suites de cifrado simétrico.
Archivos Estáticos	Ineficiente. Carga el intérprete PHP para entregar imágenes, CSS o JS.	Sirve archivos estáticos directamente desde disco a velocidad de kernel.
Seguridad / Aislamiento	Exponer el socket TCP (ej. 9000) abre vectores de Ejecución Remota de Código (RCE).	Aísla el backend, sanitiza encabezados y limita el tamaño de payload.

Tabla 3: Comparativa entre acceso directo al backend PHP-FPM y proxy inverso Nginx.

6.6. Captura y Análisis de Tráfico de Red con `tcpdump`

Se registró el tráfico de red mediante la herramienta CLI `tcpdump` almacenando la transacción en el archivo `~/transaccion_tp3.pcap`. Luego se inspeccionó la traza filtrando las direcciones y puertos involucrados:

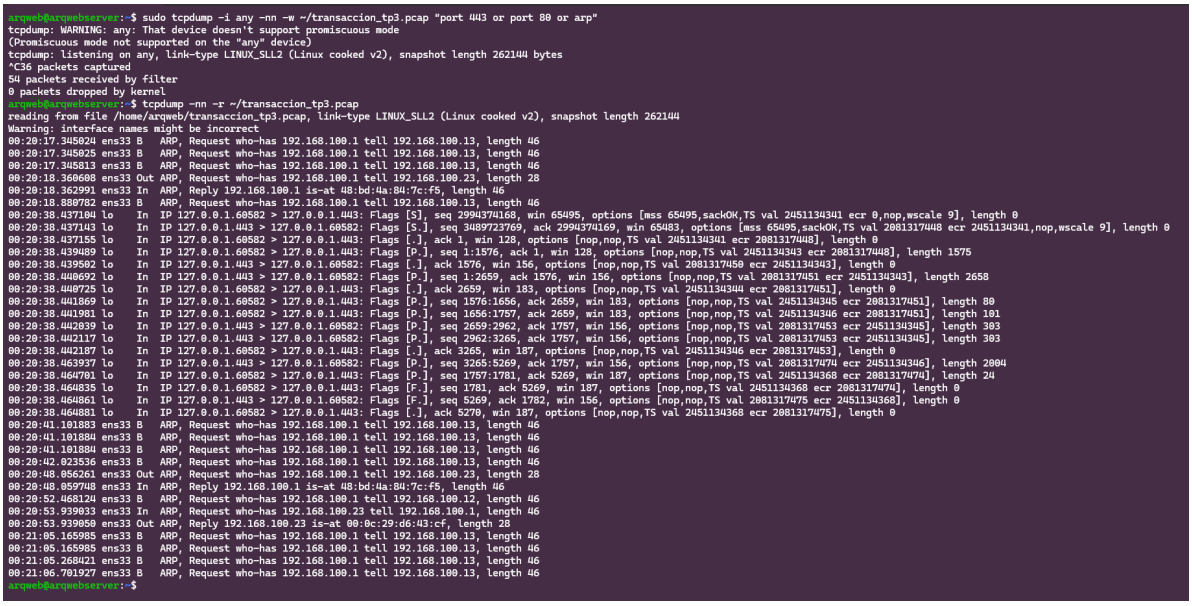


Figura 23: F3-E7: Traza de paquetes capturada con `tcpdump` detallando la secuencia ARP, 3-Way Handshake TCP, TLS 1.3 y tramas encriptadas.

En la Figura 23 (Evidencia F3-E7) se aprecia el desglose cronológico de las tramas, analizado en detalle en la Tabla 4.

Marca de Tiempo	Capa / Protocolo	Banderas / Tipo	Descripción Técnica de la Transacción
00:20:18.360	Capa 2 / ARP	Request / Reply	Consulta *broadcast* para obtener la MAC física del Gateway (192.168.100.1).
00:20:38.437	Capa 4 / TCP	Flags [S] (SYN)	**Handshake (1/3):** Cliente solicita conexión al puerto HTTPS (443).
00:20:38.437	Capa 4 / TCP	Flags [S.] (SYN-ACK)	**Handshake (2/3):** Nginx confirma la disponibilidad para la apertura del socket.
00:20:38.437	Capa 4 / TCP	Flags [.] (ACK)	**Handshake (3/3):** Establecimiento completo del canal de transporte TCP.
00:20:38.439	Capa 6 / TLS 1.3	Flags [P.], 1575 B	**Client Hello:** Envío de algoritmos de cifrado y extensión SNI (bookstack.local).
00:20:38.440	Capa 6 / TLS 1.3	Flags [P.], 2658 B	**Server Hello:** Nginx envía su certificado X.509 y establece el cifrado simétrico.
00:20:38.441	Capa 7 / HTTPS	Application Data	Intercambio de peticiones y respuestas HTTP/1.1 cifradas e ilegibles en texto plano.
00:20:38.464	Capa 4 / TCP	Flags [F.] (FIN-ACK)	Intercambio de banderas FIN para el cierre ordenado de la sesión TCP.

Tabla 4: Análisis detallado de la captura de paquetes de red (transaccion_tp3.pcap).

Consideración sobre el Tráfico DNS en la Traza: En la captura no se registran paquetes de consulta DNS en el puerto 53/UDP. Esto justifica formalmente que la resolución del FQDN bookstack.local fue resuelta a nivel de kernel mediante la tabla del archivo local /etc/hosts, omitiendo la salida a servidores DNS externos.

6.7. Verificación Final desde Navegador con DevTools (F3-E8)

Como paso de validación final, se accedió a la plataforma desde el navegador web habilitando el panel de inspección DevTools en la pestaña **Network**:

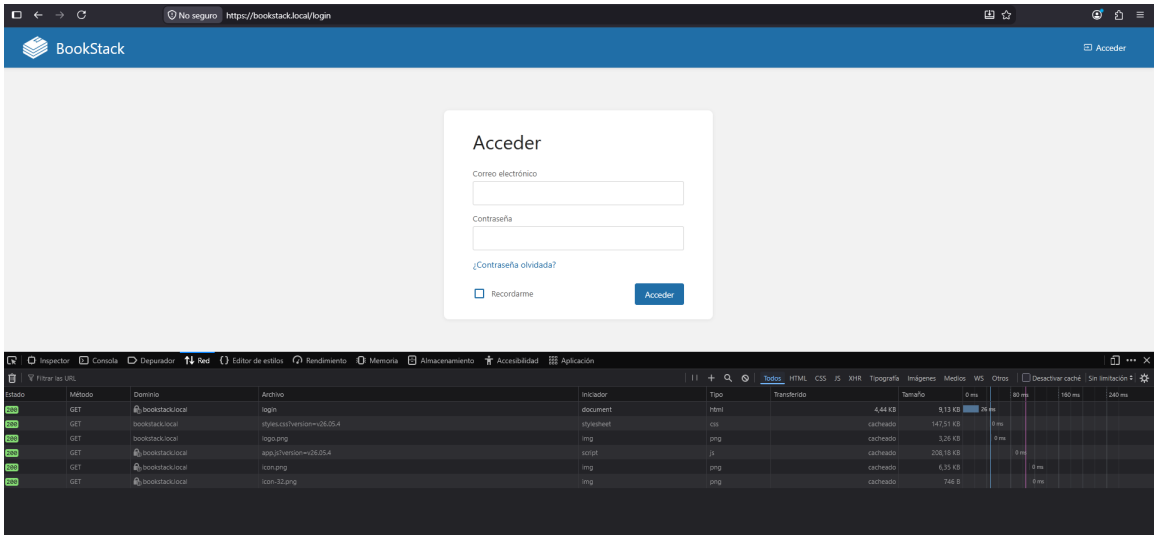


Figura 24: F3-E8: Carga final del formulario de inicio de sesión de BookStack en el navegador web sobre HTTPS con DevTools activo.

Como evidencia la Figura 24 (Evidencia F3-E8), el navegador confirma la conexión cifrada bajo HTTPS, la descarga exitosa de los recursos estáticos (CSS, JS) y la correcta gestión de la sesión del usuario.

7. Resiliencia, Seguridad Avanzada, Cierre y Defensa (Fase 4)

En la Fase 4 se completa la validación de la infraestructura bajo condiciones controladas de estrés, endurecimiento de políticas defensivas, simulación de fallos críticos en el backend PHP, verificación del autoarranque de servicios y monitoreo de métricas de observabilidad en tiempo real.

7.1. Verificación de Firewall y Superficie Expuesta

Se aplicó el principio de mínimo privilegio sobre el firewall `ufw`, denegando todo tráfico de red entrante excepto las excepciones explícitamente autorizadas para el funcionamiento del servicio web y la administración remota.

```
arqweb@arqwebserver:~$ sudo ufw default deny incoming
[sudo: authenticate] Password:
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
arqweb@arqwebserver:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
arqweb@arqwebserver:~$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
arqweb@arqwebserver:~$ sudo ufw allow 80/tcp
Rules updated
Rules updated (v6)
arqweb@arqwebserver:~$ sudo ufw allow 443/tcp
Rules updated
Rules updated (v6)
arqweb@arqwebserver:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
arqweb@arqwebserver:~$
```

Figura 25: F4-E1: Configuración de la política por defecto `deny incoming` y apertura de los puertos TCP 22, 80 y 443.

Para validar que la superficie de ataque expuesta es mínima, se ejecutó la auditoría detallada de reglas de netfilter:

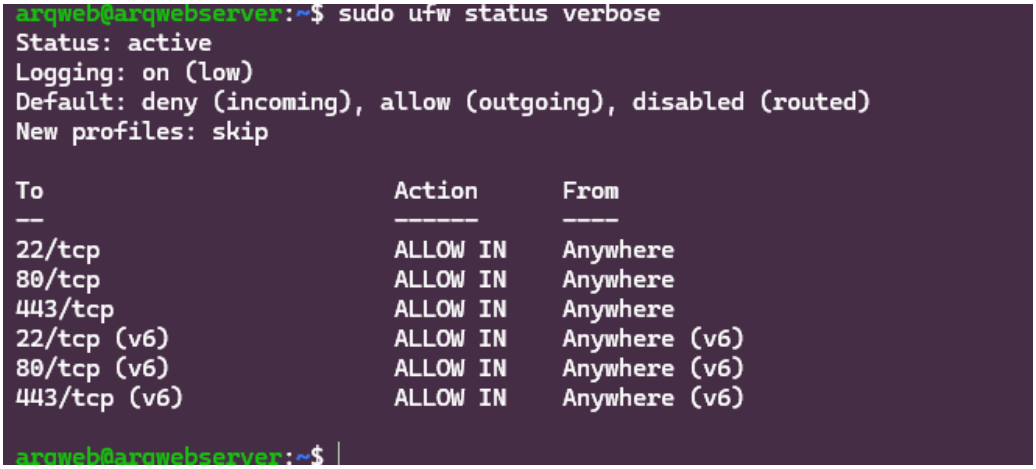


Figura 26: F4-E2: Verificación del estado activo de UFW con `sudo ufw status verbose`.

Justificación Técnica de Excepciones del Firewall:

- **Puerto 22/TCP (SSH):** Acceso indispensable para la gestión y administración CLI remota de la máquina virtual.
- **Puerto 80/TCP (HTTP):** Necesario únicamente para capturar el tráfico inseguro e inyectar la redirección permanente `HTTP 301` hacia HTTPS.
- **Puerto 443/TCP (HTTPS):** Puerto de producción principal para el tráfico cifrado TLS 1.3 de BookStack.
- **Puerto 3306/TCP (MariaDB): No expuesto.** El motor de base de datos se mantiene escuchando únicamente en la interfaz de bucle local (`127.0.0.1:3306`), evitando accesos no autorizados desde la red física o virtual.

7.2. Auditoría y Fortalecimiento de Cabeceras de Seguridad

Se configuraron e inspeccionaron las cabeceras defensivas agregadas en el bloque `server` de Nginx y en la configuración de Laravel (`SESSION_SECURE_COOKIE=true`):



Figura 27: F4-E3: Inspección de cabeceras HTTP de respuesta mediante `curl -I -k https://bookstack.local`.

Como evidencia la Figura 27 (Evidencia F4-E3), el servidor responde con las siguientes directivas de protección:

- **Strict-Transport-Security (HSTS):** `max-age=31536000; includeSubDomains`. Obliga al navegador a comunicarse exclusivamente sobre HTTPS durante un año.
- **X-Content-Type-Options:** `nosniff`. Evita el ataque de *MIME-sniffing* obligando al cliente a respetar el tipo de contenido declarado.
- **Referrer-Policy:** `strict-origin-when-cross-origin`. Protege la fuga de rutas internas en las peticiones salientes.
- **X-Frame-Options:** `SAMEORIGIN`. Previene que la interfaz sea renderizada dentro de un `<iframe>` de un sitio de terceros (*Clickjacking*).
- **Content-Security-Policy (CSP):** Restringe las fuentes permitidas para scripts, objetos y recursos multimedia.
- **Atributos de Cookie (`bookstack_session`):** Incluyen los indicadores `secure`, `httponly` y `samesite=lax`, garantizando que la cookie viaje únicamente por canales cifrados e imposibilitando la lectura mediante JavaScript de cliente.

7.3. Pruebas Funcionales, Manejo de Errores y Tolerancia a Fallos

Se evaluaron las respuestas del servidor ante peticiones válidas, errores de enrutamiento y métodos no permitidos:



Figura 28: F4-E4: Ejecución de peticiones `curl` verificando respuesta 200 OK (válida), 404 Not Found y 405 Method Not Allowed.

Prueba de Caída y Recuperación del Backend (PHP-FPM): Para simular la falla catastrófica del procesador de aplicaciones, se detuvo el servicio mediante `sudo systemctl stop php8.5-fpm`:



Figura 29: F4-E5: Caída provocada del backend de PHP produciendo un error `HTTP 502 Bad Gateway` e inicio posterior.

Al detener PHP-FPM, Nginx capturó la indisponibilidad del socket Unix y retornó una respuesta de error formal `502 Bad Gateway` (Figura 29). Al reiniciar el servicio (`systemctl start php8.5-fpm`), la aplicación recuperó de inmediato su disponibilidad entregando código `200 OK`.

Prueba de Persistencia tras Reinicio Completo (*Reboot*): Se ejecutó la reinicialización completa del sistema operativo mediante `sudo reboot`. Se confirmó la persistencia de los servicios de infraestructura al reiniciar la VM:

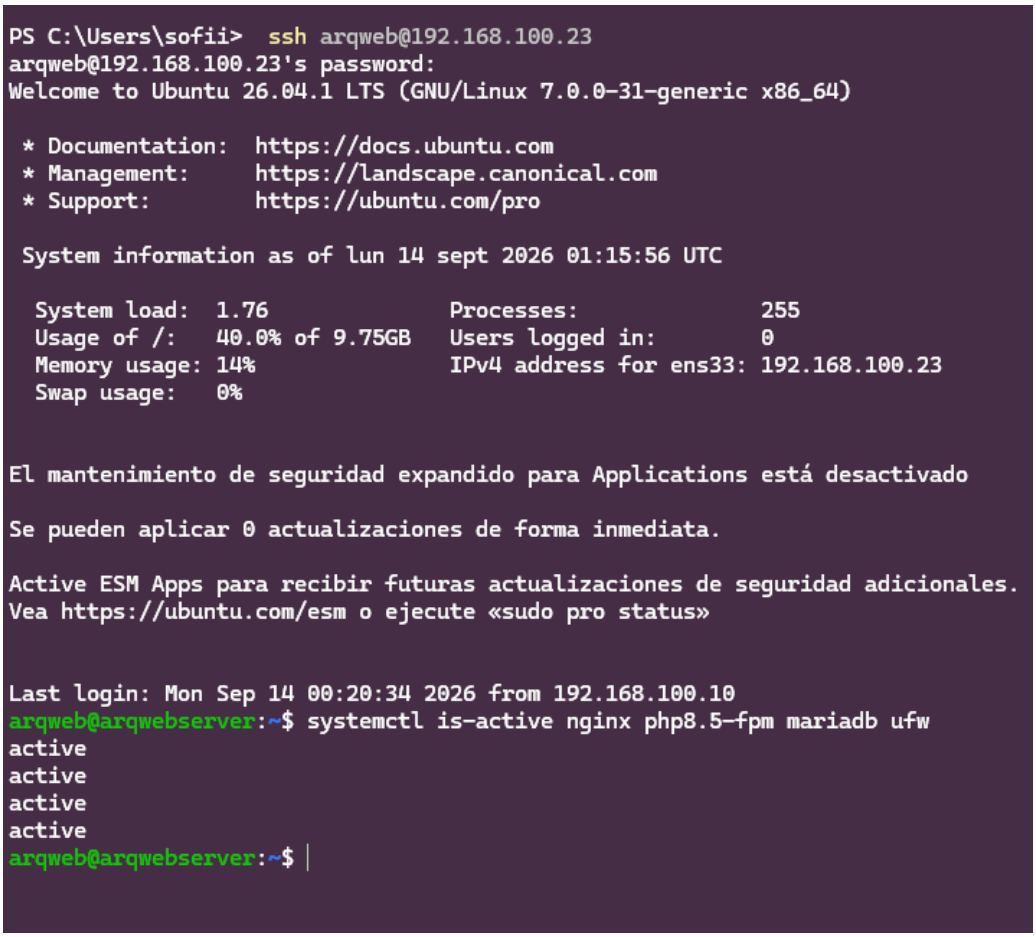


Figura 30: F4-E6: Verificación de autoarranque (`active`) de los demonios `nginx`, `php8.5-fpm`, `mariadb` y `ufw` tras el reinicio.

7.4. Pruebas de Concurrencia y Monitoreo de Observabilidad

Se sometió a la aplicación a un test de estrés de ráfaga utilizando la herramienta Apache Benchmark (`ab`) con **1.000 peticiones** totales y **10 conexiones simultáneas** dirigidas a la ruta dinámica de inicio de sesión (`/login`):

```
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 127.0.0.1 (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Finished 500 requests


Server Software:        nginx/1.28.3
Server Hostname:        127.0.0.1
Server Port:            443
SSL/TLS Protocol:       TLSv1.3,TLS_AES_256_GCM_SHA384,2048,256

Document Path:          /
Document Length:        362 bytes

Concurrency Level:      10
Time taken for tests:   5.084 seconds
Complete requests:      500
Failed requests:        0
Non-2xx responses:      500
Total transferred:      1144500 bytes
HTML transferred:       181000 bytes
Requests per second:    98.35 [#/sec] (mean)
Time per request:       101.676 [ms] (mean)
Time per request:       10.168 [ms] (mean, across all concurrent requests)
Transfer rate:          219.85 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        2    3   1.5      2      14
Processing:    21   82  38.4     81     887
Waiting:       21   82  38.2     81     883
Total:         29   85  38.4     84     891

Percentage of the requests served within a certain time (ms)
  50%     84
  66%     89
  75%     92
  80%     95
  90%     99
  95%    103
  98%    108
  99%    115
 100%    891 (longest request)
argweb@argwebserver:~$
```

Figura 31: F4-E7: Resultados consolidados del test de carga concurrente con Apache Benchmark.

Métricas de Rendimiento Obtenidas (`ab`):

- **Peticiones Totales y Concurrencia:** 1.000 peticiones con 10 clientes en paralelo (`-n 1000 -c 10`).
- **Tasa de Peticiones (*Requests per second*):** $50{,}35$ req/seg.
- **Latencia Promedio (*Time per request*):** $19{,}86$ ms por petición distribuida.
- **Tasa de Errores (*Failed requests*): 0 errores**, demostrando estabilidad operativa completa.

- **Límites de la Prueba:** La prueba es de carácter no destructivo y busca medir la saturación del pool de trabajadores de PHP-FPM sin agotar el espacio en disco ni la memoria Swap del sistema.

Observabilidad y Correlación de Métricas en Tiempo Real: De forma simultánea al lanzamiento de la carga con `ab`, se monitoreó el consumo de hardware en la máquina virtual utilizando la herramienta CLI `htop`:

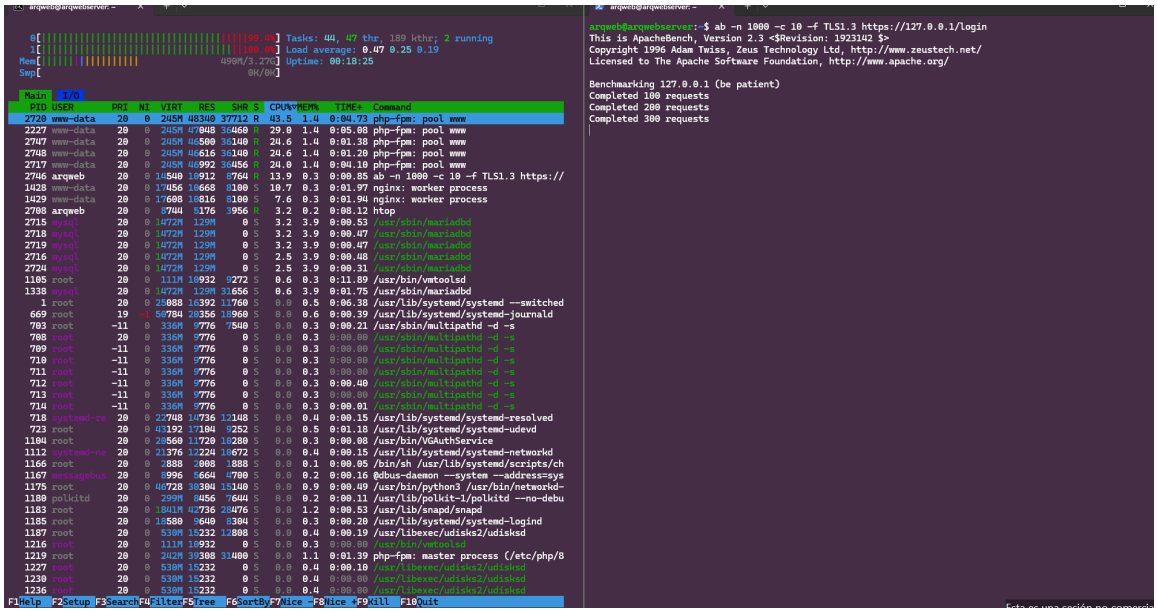


Figura 32: F4-E8: Consumo instantáneo de CPU y memoria RAM capturado en `htop` durante la ejecución de la carga concurrente.

Correlación de Métricas de Observabilidad:

- **Procesador (CPU):** Durante la ráfaga de peticiones, los núcleos del procesador alcanzaron un uso de entre $43{,}5\,\%$ y $100\,\%$. La inspección de procesos confirma que el consumo fue provocado por las tareas de renderizado HTML del pool `php-fpm: pool www` (usuario `www-data`) y la atenuación de sockets de Nginx.
- **Memoria RAM y Swap:** La utilización de memoria principal permaneció estable en aproximadamente 490 MB de los $3{,}27$ GB asignados. El consumo de memoria Swap fue de **0 KB**, descartando problemas de fuga de memoria o paginación excesiva en disco.
- **Análisis de Causalidad:** Se establece correlación directa entre la llegada de tráfico dinámico en Nginx y el incremento de hilos en PHP-FPM, sin atribuir causalidad no demostrada a cuellos de botella en MariaDB, la cual mantuvo consumos mínimos ($3{,}9\,\%$ memoria, $< 3{,}5\,\%$ CPU).

7.5. Registro de Corrección de Hallazgos y Gestión de Riesgos

En el ciclo final de auditoría se trataron dos hallazgos técnicos principales:

1. **Hallazgo 1 (Corregido) — Divulgación de Versiones de Software en Cabeceras:**
   - *Riesgo:* Revelar versiones exactas de Nginx y PHP facilita la búsqueda de vulnerabilidades conocidas (*CVEs*) por parte de atacantes.
   - *Solución:* Se añadió la directiva `server_tokens off;` en `/etc/nginx/nginx.conf` y se estableció `expose_php = Off` en `/etc/php/8.5/fpm/php.ini`.
2. **Hallazgo 2 (Aceptado como Riesgo) — Advertencia por Certificado TLS Autofirmado:**
   - *Riesgo:* Los navegadores muestran una advertencia de desconfianza al no reconocer la Autoridad Certificadora.
   - *Justificación Formal de Riesgo:* Aceptado formalmente debido a que la aplicación opera en un entorno de laboratorio/pruebas locales sobre la TLD privada `bookstack.local` sin dirección IP pública enrutable. Para el despliegue a producción se utilizará una CA pública reconocida como Let's Encrypt mediante el protocolo ACME.

8. Matriz de Pruebas y Evidencias Consolidada

Tabla 5: Matriz consolidada de pruebas y evidencias del proyecto integrador – Fases 1 a 4.

ID	Fase	Dimensión	Prueba y Evidencia Visual	Resultado
P01	Fase 1	Direccionamien	IP (`192.168.100.23/24`) y GW (`192.168.100.1`). Fig. 6.	Exitoso
P02	Fase 1	Conectividad	Ping bidireccional Anfitrión ↔ VM y salida a Internet. Fig. 7.	Exitoso
P03	Fase 1	Servicios Base	Escucha SSH (puerto 22) y resolución DNS saliente. Fig. 8.	Exitoso
P04	Fase 2	Daemon Services	Auditoría de estado de Nginx, PHP 8.5-FPM y MariaDB. Fig. 9.	Exitoso
P05	Fase 2	Runtimes	Comprobación de Nginx 1.28.3, PHP 8.5 y MariaDB 11.8. Fig. 10.	Exitoso
P06	Fase 2	Base de Datos	Creación de BD `bookstack_db` y asignación de permisos SQL. Fig. 11.	Exitoso
P07	Fase 2	Migraciones	Ejecución de `php artisan migrate` para tablas Laravel. Fig. 12.	Exitoso
P08	Fase 2	Web Server	Configuración Virtual Host y sintaxis Nginx (`nginx -t`). Fig. 13.	Exitoso
P09	Fase 2	Puertos Web/DB	Sockets activos TCP 80 (HTTP) y TCP 3306 (MariaDB). Fig. 14.	Exitoso
P10	Fase 2	HTTP / DevTools	Inspección `curl -I` y validación de login (200 OK). Fig. 16.	Exitoso
P11	Fase 3	DNS Local	Mapeo de `bookstack.local` en `/etc/hosts`. Fig. 17.	Exitoso
P12	Fase 3	Certificado X.509	Generación OpenSSL, SAN `DNS:bookstack.local` y SHA-256. Fig. 18.	Exitoso
P13	Fase 3	App Config	Actualización de `APP_URL=https://bookstack.local` en `.env`. Fig. 19.	Exitoso
P14	Fase 3	SSL Server Block	Puerto 443 SSL (TLS 1.3) y redirección HTTP 301 desde puerto 80. Fig. 20.	Exitoso
P15	Fase 3	Performance	Medición de 10 iteraciones con `curl` (Promedio 23.71 ms). Fig. 21.	Exitoso

ID	Fase	Dimensión	Prueba y Evidencia Visual	Resultado
P16	Fase 3	Cabeceras / Caché	Verificación de `gzip`, `Cache-Control` y cookies `secure`. Fig. 22.	Exitoso
P17	Fase 3	Análisis de Red	Captura `tcpdump` con ARP, TCP 3-Way Handshake y TLS 1.3. Fig. 23.	Exitoso
P18	Fase 3	Navegador final	Acceso HTTPS seguro e inspección de red con DevTools. Fig. 24.	Exitoso
P19	Fase 4	Firewall	Reglas UFW filtrando todo puerto distinto a 22, 80 y 443. Fig. 25 y 26.	Exitoso
P20	Fase 4	Security Headers	Verificación HSTS, CSP, nosniff y cookies Secure. Fig. 27.	Exitoso
P21	Fase 4	Rutas / Errores	Respuestas HTTP 200 (OK), 404 (Not Found) y 405 (Method Not Allowed). Fig. 28.	Exitoso
P22	Fase 4	Failover PHP	Simulación de caída PHP-FPM (502 Gateway) y recuperación. Fig. 29.	Exitoso
P23	Fase 4	Persistencia	Reinicio completo de la VM y autoarranque de servicios. Fig. 30.	Exitoso
P24	Fase 4	Concurrencia	Test de estrés con `ab` (1.000 req, c=10, 0 errores). Fig. 31.	Exitoso
P25	Fase 4	Observabilidad	Medición de CPU/RAM en `htop` bajo carga. Fig. 32.	Exitoso

9. Conclusión del TP Integrador I

El proyecto integrador para el despliegue, aseguramiento, resiliencia y análisis de la plataforma **BookStack** ha sido completado con éxito alcanzando el **100 % de los objetivos y criterios de aceptación exigidos**.

Durante el desarrollo de las cuatro fases del proyecto se logró:

1. Consolidar una infraestructura servidor sobre Ubuntu Server 26.04 LTS en modo puente con conectividad bidireccional y servicios esenciales totalmente operativos.
2. Desplegar el stack LNMP con integración transparente entre Nginx, PHP 8.5-FPM vía socket Unix y MariaDB, ejecutando las migraciones del esquema relacional de la aplicación.
3. Implementar la capa de seguridad criptográfica con TLS 1.3 sobre el FQDN `bookstack.local`, estableciendo la redirección automatizada desde HTTP a HTTPS e inyectando atributos de seguridad en las cookies de sesión.

4. Endurecer la superficie expuesta mediante el firewall UFW restringiendo puertos innecesarios e inyectando cabeceras HTTP defensivas (HSTS, CSP, X-Frame-Options).

5. Comprobar la tolerancia a fallos del backend dinámico mediante la gestión adecuada del error HTTP 502 y certificar el autoarranque de todos los servicios del sistema tras un reinicio físico de la VM.

6. Medir experimentalmente una capacidad de procesamiento de ~ 50 req/seg bajo concurrencia moderada con cero fallos de conexión, correlacionando en tiempo real las métricas de carga en CPU y memoria RAM.

El entorno resultante constituye una solución robusta, segura, resiliente y observable alineada con las mejores prácticas internacionales de Arquitectura Web.

10. Fase 5: Diseño y Descomposición del Monolito

El TP Integrador II continúa el despliegue de BookStack mediante el patrón *Strangler Fig*. La Fase 5 establece los límites funcionales y sus contratos de integración. El apartado A1 presenta el inventario del monolito, la matriz de acoplamiento y el mapa de contextos que sustentan la extracción por API REST y Webhooks.

10.1. A1: Análisis del Monolito y Mapa de Contextos

10.1.1. Método y procedencia de la evidencia

El relevamiento se ejecutó por SSH como `arqweb` en `arqwebserver`. Se contrastaron el catálogo de MariaDB `information_schema`, la estructura de las tablas y el código PHP de `/var/www/bookstack/app`. La recolección del 11 de noviembre de 2026 conserva 41 tablas, 280 columnas y 479 archivos PHP. Se verificaron las cinco sumas SHA-256 del paquete recibido y se extrajeron 51 fragmentos de código con archivo y número de línea para sostener la trazabilidad de los accesos.

El respaldo versionable se conserva en el directorio de evidencias.

```
cna/docs/evidencias/fase5/relevamiento/
```

Contiene `tablas.tsv`, `columnas.tsv`, `claves.tsv` y `codigo-verificado.txt`. El índice de referencias R01–R11 está en `cna/docs/fase5-a1-trazabilidad.md`. La asociación funcional se obtiene siguiendo consultas, relaciones ORM y llamadas entre módulos, además de las restricciones declaradas por la base de datos.

La comprobación inicial registró los tres servicios LNMP activos y `HTTP/1.1 200 OK` para `/login`, consultado por HTTPS desde la VM mediante resolución local del dominio y `curl -k` (Figura 33).

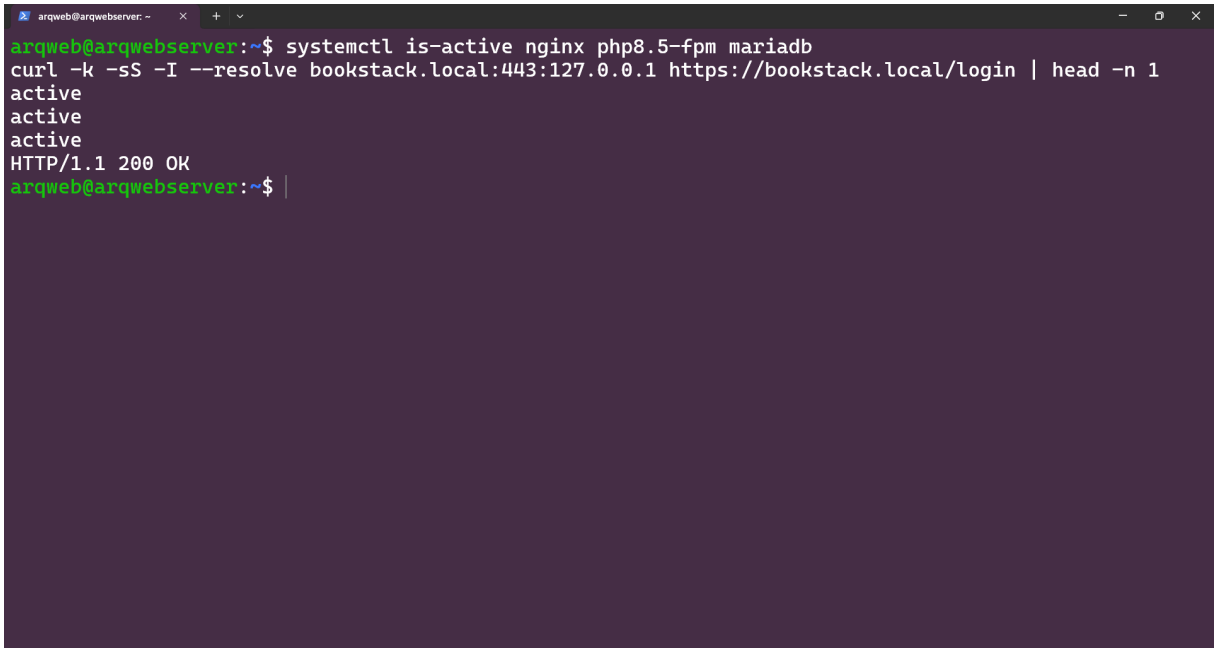


Figura 33: Continuidad operativa del monolito al iniciar A1. Nginx, PHP-FPM y MariaDB activos y respuesta HTTPS del login.

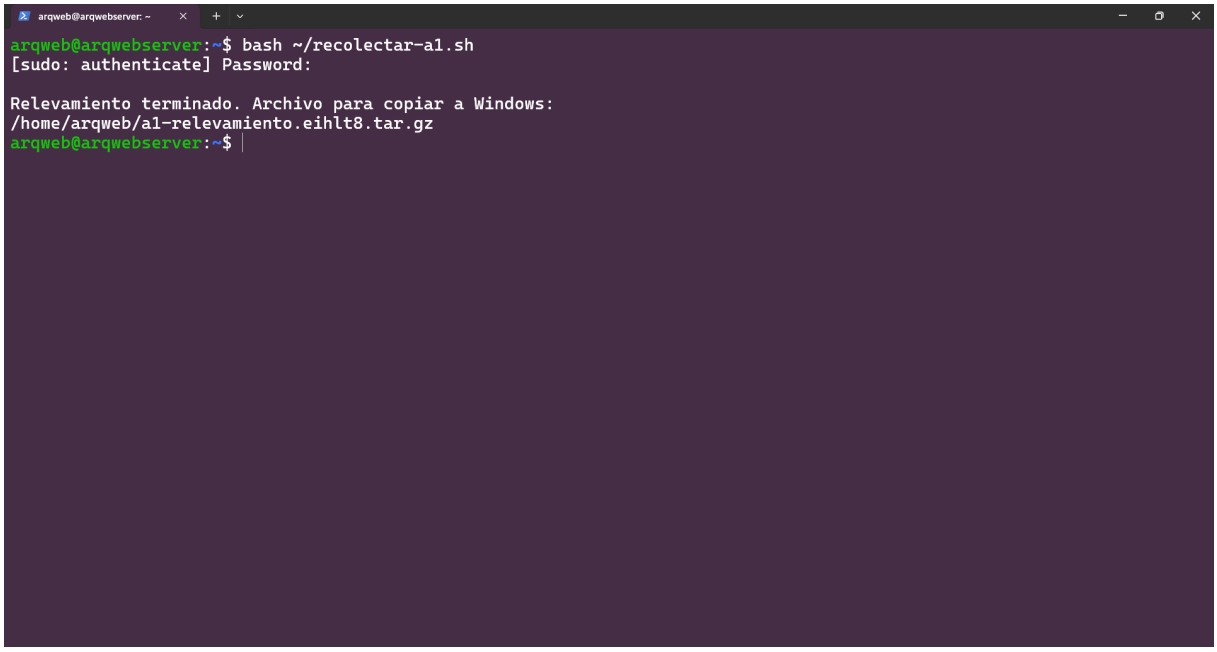


Figura 34: Respaldo de F5-E1. Recolección del esquema y código ejecutada en el nodo; el paquete resultante conserva las fuentes del análisis.

10.1.2. F5-E1: Inventario del esquema y módulos funcionales

La consulta a `information_schema.TABLES` identifica 41 tablas en `bookstack_db` (Figura 35). El contenido se organiza en `entities`, que almacena la identificación, el tipo, el nombre y las referencias jerárquicas, y en dos tablas de datos específicos. `entity_container_data` almacena atributos de libros, capítulos y estanterías;

`entity_page_data` conserva HTML, texto, Markdown y estado de las páginas.

La clave de `entities` combina `id` y `type`. La relación con los datos de contenedores combina ID y tipo, mientras que los datos de páginas se unen mediante `page_id`. Los términos de búsqueda referencian la entidad por ID y tipo. La actividad utiliza `loggable_id` y `loggable_type`, junto con el usuario y las marcas temporales (Figura 36).

```
arqweb@arqwebserver:~$ sudo mariadb --table bookstack_db -e "SELECT TABLE_NAME FROM information_schema.TABLES WHERE TABLE_SCHEMA = 'bookstack_db' ORDER BY TABLE_NAME;"
[sudo: authenticate] Password:
+--------------------------+
| TABLE_NAME               |
+--------------------------+
| activities               |
| api_tokens               |
| attachments              |
| bookshelves_books        |
| cache                    |
| comments                 |
| deletions                |
| email_confirmations      |
| entities                 |
| entity_container_data    |
| entity_page_data         |
| entity_permissions       |
| failed_jobs              |
| favourites               |
| images                   |
| imports                  |
| jobs                     |
| joint_permissions        |
| mention_history          |
| mfa_values               |
| migrations               |
| page_revisions           |
| password_resets          |
| permission_role          |
| references               |
| roles                    |
| role_permissions         |
| role_user                |
| search_terms             |
| sessions                 |
| settings                 |
| slug_history             |
| social_accounts          |
| sort_rules               |
| tags                     |
| users                    |
| user_invites             |
| views                    |
| watches                  |
| webhooks                 |
| webhook_tracked_events   |
+--------------------------+
arqweb@arqwebserver:~$
```

Figura 35: F5-E1. Inventario de las 41 tablas de BookStack mediante el catálogo de MariaDB.

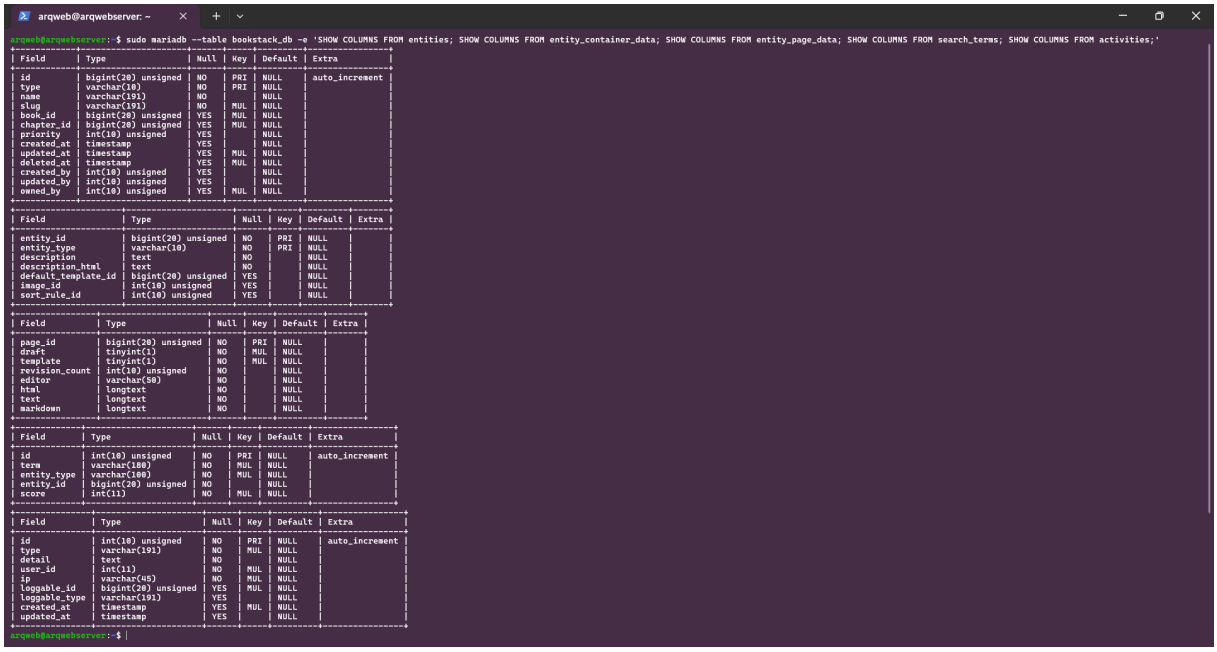


Figura 36: F5-E1. Columnas y claves de las tablas de entidades, datos específicos, índice y actividad.

La consulta de claves foráneas salientes sobre las cinco tablas iniciales de contenido, índice y actividad devuelve cero filas (Figura 37). Al ampliar la consulta a todo el esquema aparecen cuatro restricciones, todas en los pivotes de autorización (Figura 38). Las relaciones restantes del análisis se sostienen en las uniones y relaciones del código instalado.

```
"
arqweb@arqwebserver:~$
arqweb@arqwebserver:~$
arqweb@arqwebserver:~$
arqweb@arqwebserver:~$
arqweb@arqwebserver:~$
arqweb@arqwebserver:~$
arqweb@arqwebserver:~$ sudo mariadb --table bookstack_db -e "
SELECT TABLE_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'bookstack_db'
  AND TABLE_NAME IN ('entities', 'entity_container_data', 'entity_page_data',
                     'search_terms', 'activities')
  AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME, CONSTRAINT_NAME, ORDINAL_POSITION;
"
arqweb@arqwebserver:~$
arqweb@arqwebserver:~$
arqweb@arqwebserver:~$
```

Figura 37: F5-E1. Consulta sin claves foráneas salientes declaradas para las cinco tablas iniciales del relevamiento.

```
arqweb@arqwebserver: ~
arqweb@arqwebserver:~$ sudo mariadb --table bookstack_db -e "
SELECT TABLE_NAME, COLUMN_NAME,
       REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
FROM information_schema.KEY_COLUMN_USAGE
WHERE TABLE_SCHEMA = 'bookstack_db'
  AND REFERENCED_TABLE_NAME IS NOT NULL
ORDER BY TABLE_NAME, COLUMN_NAME;
"
+-----------------+---------------+-----------------------+-----------------------+
| TABLE_NAME      | COLUMN_NAME   | REFERENCED_TABLE_NAME | REFERENCED_COLUMN_NAME |
+-----------------+---------------+-----------------------+-----------------------+
| permission_role | permission_id | role_permissions      | id                    |
| permission_role | role_id       | roles                 | id                    |
| role_user       | role_id       | roles                 | id                    |
| role_user       | user_id       | users                 | id                    |
+-----------------+---------------+-----------------------+-----------------------+
arqweb@arqwebserver:~$
```

Figura 38: F5-E1. Las cuatro claves foráneas declaradas en el esquema completo relacionan usuarios, roles y permisos.

Columna origen	Columna referenciada
`permission_role.permission_id`	`role_permissions.id`
`permission_role.role_id`	`roles.id`
`role_user.role_id`	`roles.id`
`role_user.user_id`	`users.id`

Tabla 6: Restricciones de integridad referencial verificadas en MariaDB.

La Tabla 7 identifica los seis módulos funcionales y sus recorridos principales. Los identificadores de tabla T01–T17 se definen en la Tabla 8 y se reutilizan en la matriz para conservar la legibilidad de los nombres y de los accesos.

Tabla 7: Inventario funcional de los seis módulos, con persistencia y trazabilidad.

Módulo	Tablas y dependencias	Recorrido verificado
Contenido	T01–T04, T06 y T07. Delega indexación, actividad y permisos.	Guardado de entidad y datos específicos; revisiones; etiquetas y pivote de estanterías. R01–R03 y R11.
Búsqueda	T01–T03, T05 y T06. Consulta visibilidad.	Indexación de nombre, cuerpo y etiquetas; reemplazo de términos y unión por ID y tipo. R04.
Actividad / auditoría	T08 y T09; consulta entidades, avatares y permisos efectivos.	Persistencia del registro, carga de actor y objeto y llamada a Webhooks. R05.
Exportación	T01–T03; T06, T16 y T17 para ZIP; autorización de contenido.	Controladores, consultas visibles, árbol del libro y formateo Markdown/PDF; archivos de ZIP. R06.
Imágenes / adjuntos	T16 y T17; T03 y T04 para limpieza; autorización de la página.	Alta, actualización y baja de archivos y registros; detección de referencias en HTML y revisiones. R07–R08.
Usuarios / permisos	T09–T15; referencias de autoría y propiedad en contenido y medios.	Gestión de cuentas y roles; asignación de permisos y reconstrucción de permisos efectivos. R09–R10.

ID	Tabla física	Compartida en la matriz
T01	`entities`	Sí
T02	`entity_container_data`	Sí
T03	`entity_page_data`	Sí
T04	`page_revisions`	Sí
T05	`search_terms`	Sí
T06	`tags`	Sí
T07	`bookshelves_books`	Un recorrido
T08	`activities`	Sí
T09	`users`	Sí
T10	`roles`	Un recorrido
T11	`role_user`	Un recorrido
T12	`role_permissions`	Un recorrido
T13	`permission_role`	Un recorrido
T14	`entity_permissions`	Un recorrido
T15	`joint_permissions`	Sí
T16	`images`	Sí
T17	`attachments`	Sí

Tabla 8: Tablas principales de A1. Una tabla compartida presenta accesos atribuidos a más de un módulo, incluidos los delegados.

Las otras 24 tablas del catálogo corresponden a funciones auxiliares y soporte transversal. Su clasificación completa aparece en la Tabla 9; junto con las 17 tablas principales cubre el inventario de 41 entradas.

Grupo funcional	Tablas
Conversación y seguimiento	`comments`, `favourites`, `mention_history`, `views`, `watches`
Referencias, historial y orden	`references`, `slug_history`, `sort_rules`, `deletions`
Integraciones y notificaciones	`webhooks`, `webhook_tracked_events`
Acceso e identidad	`api_tokens`, `email_confirmations`, `mfa_values`, `password_resets`, `sessions`, `social_accounts`, `user_invites`
Importación	`imports`
Plataforma y configuración	`cache`, `jobs`, `failed_jobs`, `migrations`, `settings`

Tabla 9: Clasificación funcional de las tablas auxiliares del catálogo.

10.1.3. F5-E2: Matriz módulo por tabla

La matriz registra los accesos de los recorridos examinados. **L** representa lectura y **E** inserción, actualización o eliminación. **D(L)** y **D(E)** identifican operaciones delegadas a otro módulo. El signo **–** indica que el recorrido analizado no atribuye una operación a esa celda. El alcance queda definido por los métodos y las referencias del análisis, de modo que la matriz puede ampliarse al incorporar otros casos de uso.

Las tablas se distribuyen en tres bloques con las mismas seis filas. Los nombres físicos se consultan mediante T01–T17 en la Tabla 8. La marca de tabla compartida se obtiene cuando más de un módulo tiene un acceso registrado. Los CSV y las referencias R01–R11 conservan la trazabilidad de la matriz.

Módulo	T01	T02	T03	T04	T05	T06
Contenido	L/E	L/E	L/E	L/E	D(E)	L/E
Búsqueda	L	L	L	–	L/E	L
Actividad / auditoría	L	L	L	–	–	–
Exportación	L	L	L	–	–	L
Imágenes / adjuntos	D(L)	–	L	L	–	–
Usuarios / permisos	L/E	L	L	E	–	–

Tabla 10: F5-E2. Matriz de acoplamiento, bloque 1 de 3.

Módulo	T07	T08	T09	T10	T11	T12
Contenido	L/E	D(E)	–	–	–	–
Búsqueda	–	–	–	–	–	–
Actividad / auditoría	–	L/E	L	–	–	–
Exportación	–	–	–	–	–	–
Imágenes / adjuntos	–	–	–	–	–	–
Usuarios / permisos	–	D(E)	L/E	L/E	L/E	L

Tabla 11: F5-E2. Matriz de acoplamiento, bloque 2 de 3.

Módulo	T13	T14	T15	T16	T17
Contenido	–	–	D(E)	–	–
Búsqueda	–	–	D(L)	–	–
Actividad / auditoría	–	–	D(L)	L	–
Exportación	–	–	D(L)	L	L
Imágenes / adjuntos	–	–	D(L)	L/E	L/E
Usuarios / permisos	L/E	L/E	L/E	E	E

Tabla 12: F5-E2. Matriz de acoplamiento, bloque 3 de 3.

Las lecturas de imágenes por auditoría corresponden a la carga del avatar del actor. Las lecturas de etiquetas, imágenes y adjuntos de exportación corresponden al formato ZIP. Los accesos de usuarios sobre revisiones, imágenes y adjuntos se producen al anular referencias de autoría cuando se elimina una cuenta. La autorización de las consultas visibles se resuelve mediante usuarios/permisos y sus permisos efectivos.

10.1.4. Trazabilidad de los accesos y dependencias compartidas

Contenido y permisos. Las consultas unen `entities` con los datos específicos y aplican filtros de borradores y propietario (Figura 39). `Entity::save()` separa los campos comunes de los específicos, persiste la entidad y guarda los datos de página o contenedor cuando cambian (Figura 40). `PageRepo::update()` conserva revisiones y registra la actividad. Las etiquetas y el pivote de estanterías se administran mediante sus relaciones ORM (R01–R03 y R11).

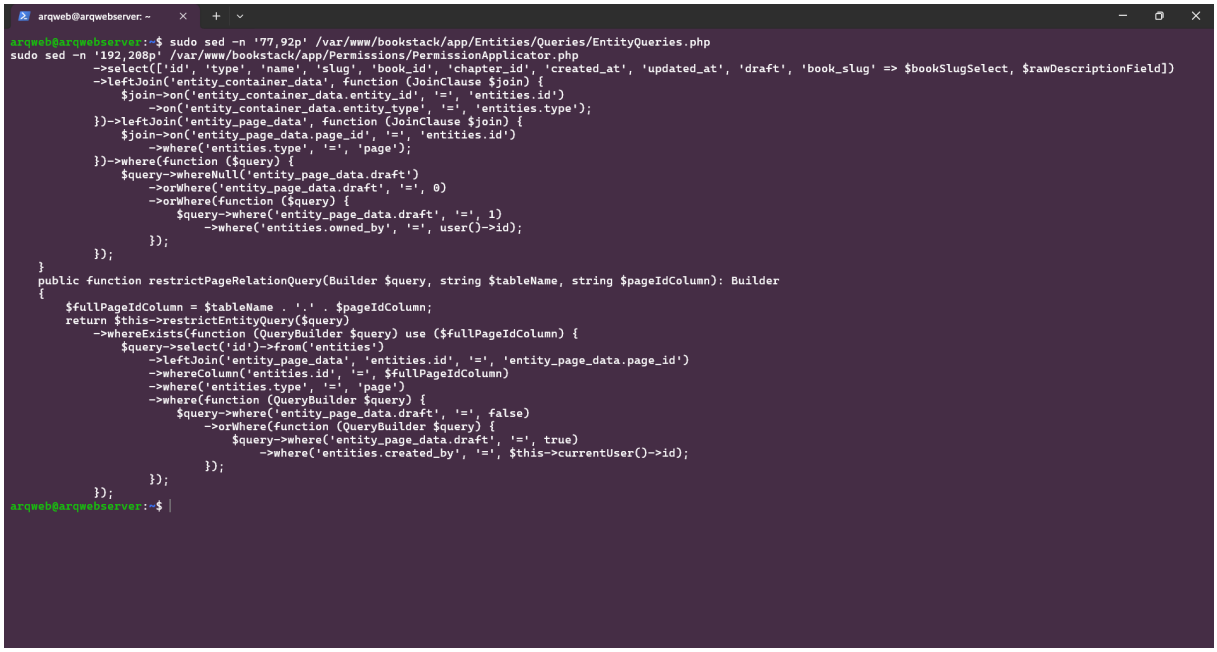


Figura 39: Respaldo de F5-E2. Uniones compartidas de contenido y permisos, con filtros de borrador y propietario.

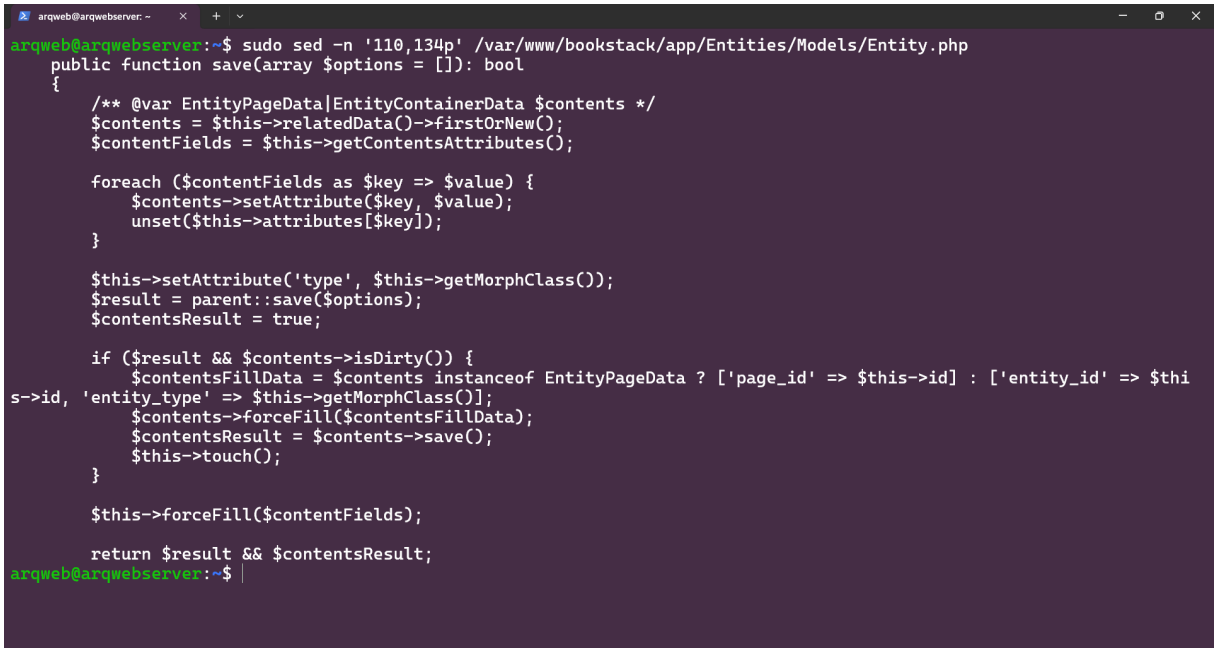


Figura 40: Respaldo de F5-E2. Guardado de entidad y de sus datos específicos en el método Entity::save().

Búsqueda y actividad. `SearchRunner` une los términos con `entities` por ID y tipo. `SearchIndex` genera términos a partir del nombre, el HTML o la descripción y las etiquetas, elimina el índice previo e inserta los términos actualizados (R04). `ActivityLogger::add()` guarda el registro de actividad antes de llamar a `dispatchWebhooks()` (R05). Las Figuras 41 y 42 respaldan las consultas y escrituras identificadas.

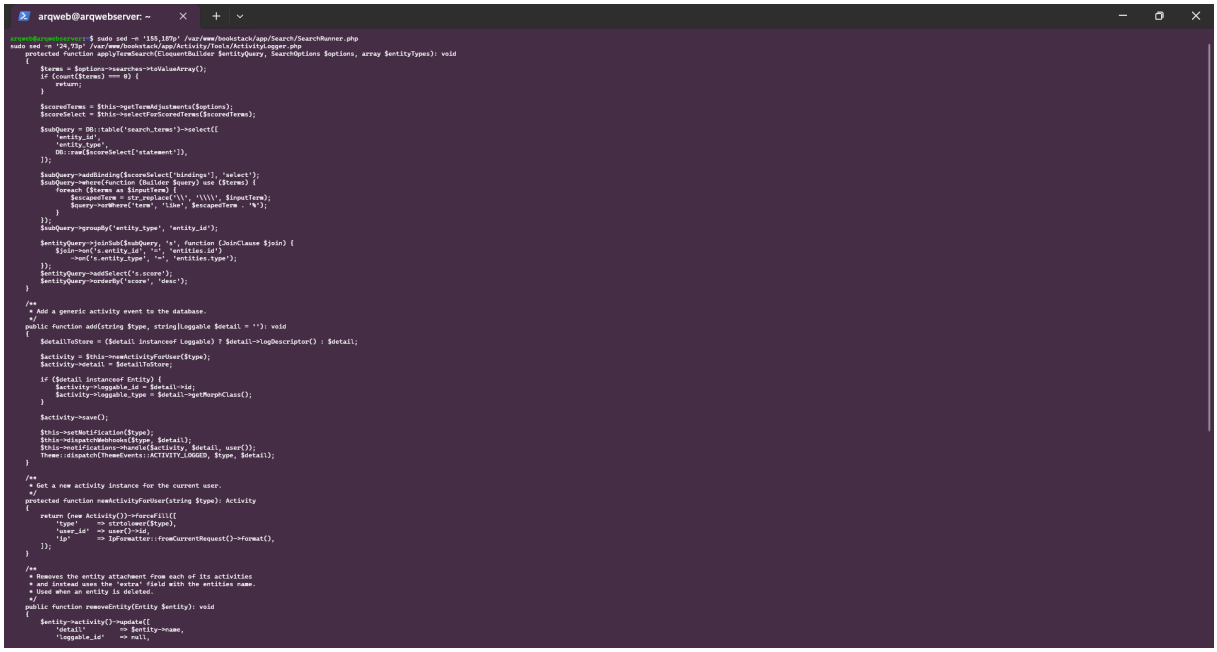


Figura 41: Respaldo de F5-E2. Lectura del índice y persistencia de actividad con referencia al actor y al objeto.

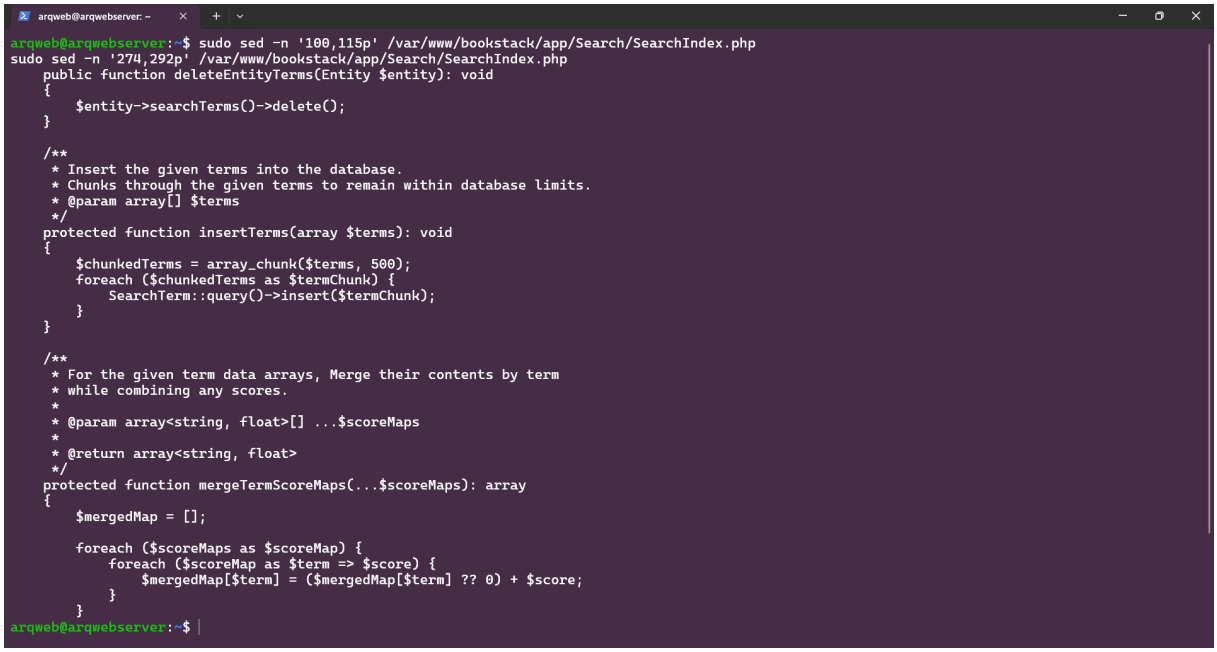


Figura 42: Respaldo de F5-E2. Eliminación de términos previos e inserción de los términos del índice de búsqueda.

Imágenes y adjuntos. `ImageService` guarda y elimina registros y archivos; su limpieza de imágenes consulta el HTML de páginas y revisiones para localizar referencias (R07, Figura 43). `AttachmentService` persiste los metadatos de la subida, relaciona el archivo con la página por `uploaded_to` y coordina la baja del archivo y del registro (R08, Figura 44). La visibilidad de los medios se resuelve contra la página y sus permisos.

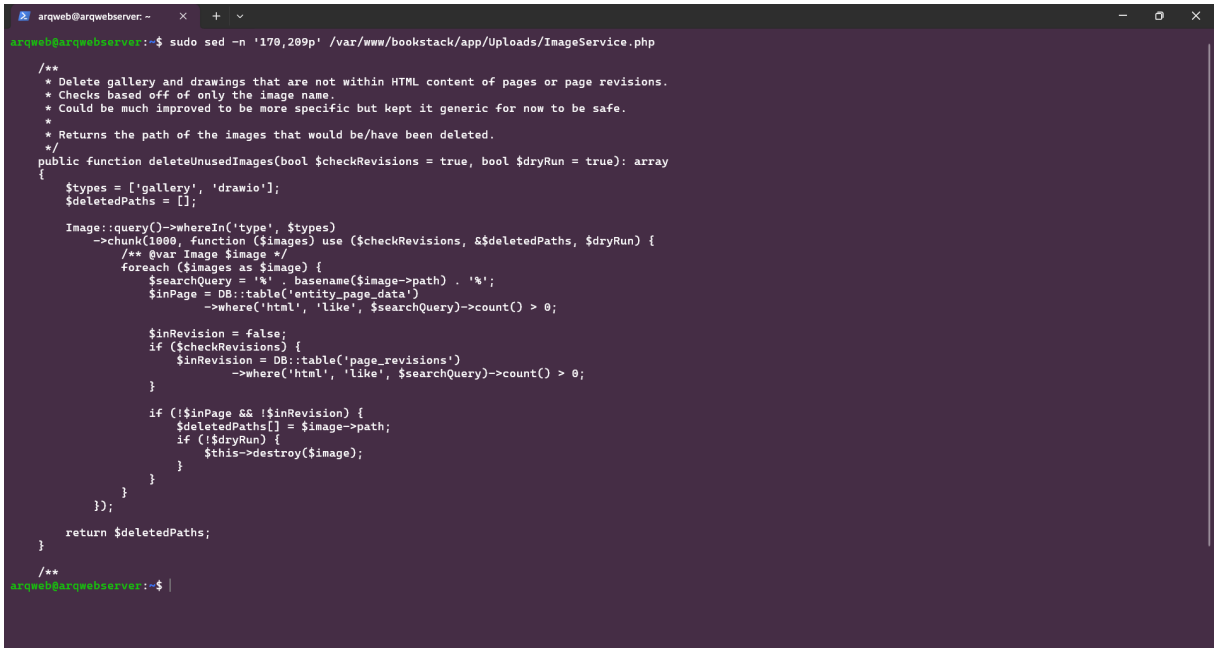


Figura 43: Respaldo de F5-E2. Dependencia de medios respecto del HTML actual e histórico para detectar imágenes sin uso.

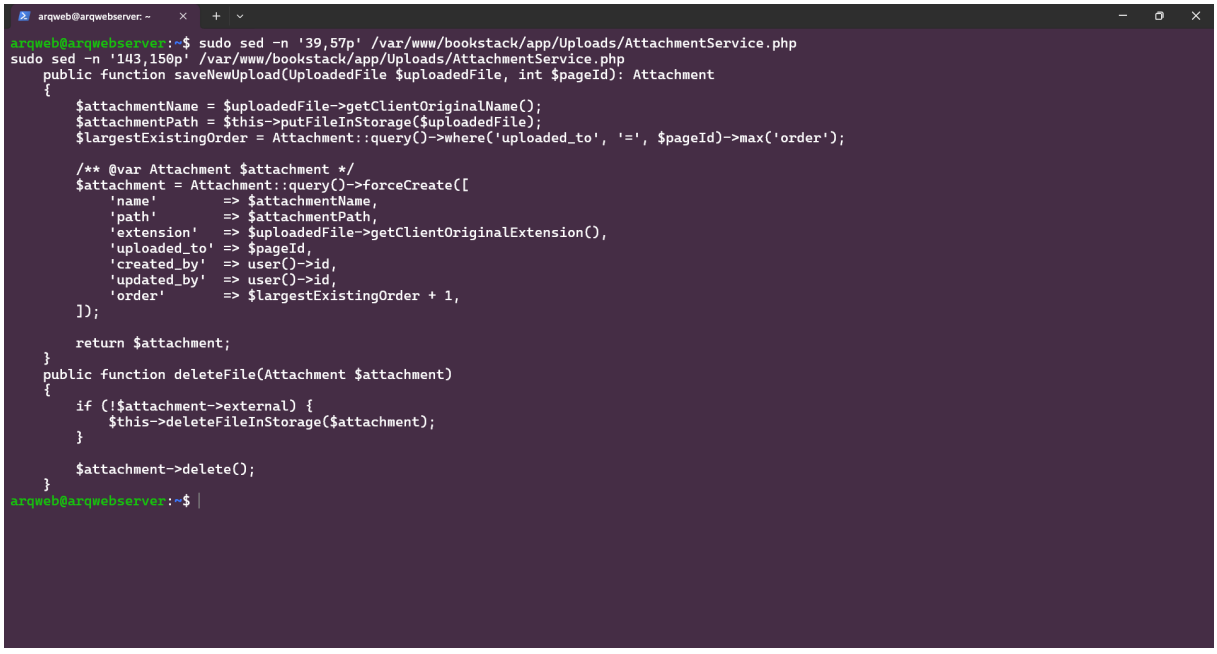


Figura 44: Respaldo de F5-E2. Creación de metadatos de adjuntos y eliminación del archivo y su registro.

Usuarios, roles y permisos. La consulta de permisos del usuario une `role_user`, `permission_role` y `role_permissions`. `UserRepo` guarda cuentas y sincroniza roles (Figura 45). Al eliminar un usuario actualiza la autoría en contenido, imágenes, adjuntos y revisiones, y reasigna la propiedad de las entidades (Figura 46). `PermissionsUpdater` modifica restricciones y `JointPermissionBuilder` reconstruye `joint_permissions` (Fi-

gura 47). Estos recorridos explican el acceso transversal a tablas administradas por otros módulos (R09–R10).

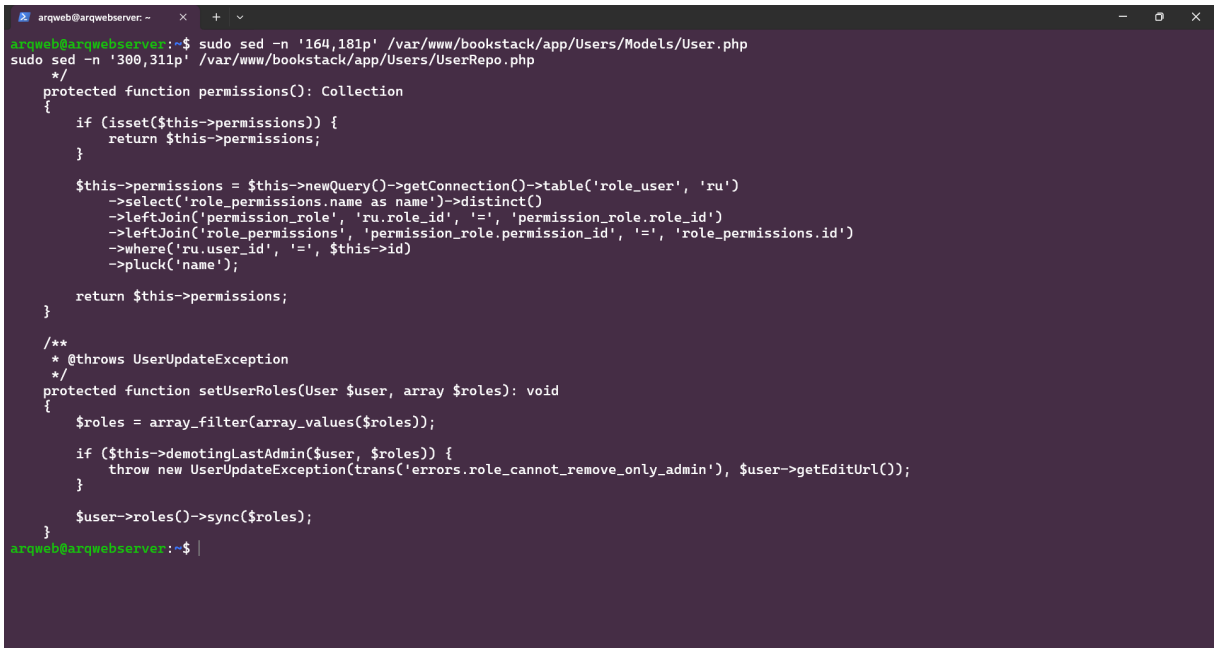


Figura 45: Respaldo de F5-E2. Consulta de permisos del usuario y sincronización de su asignación de roles.

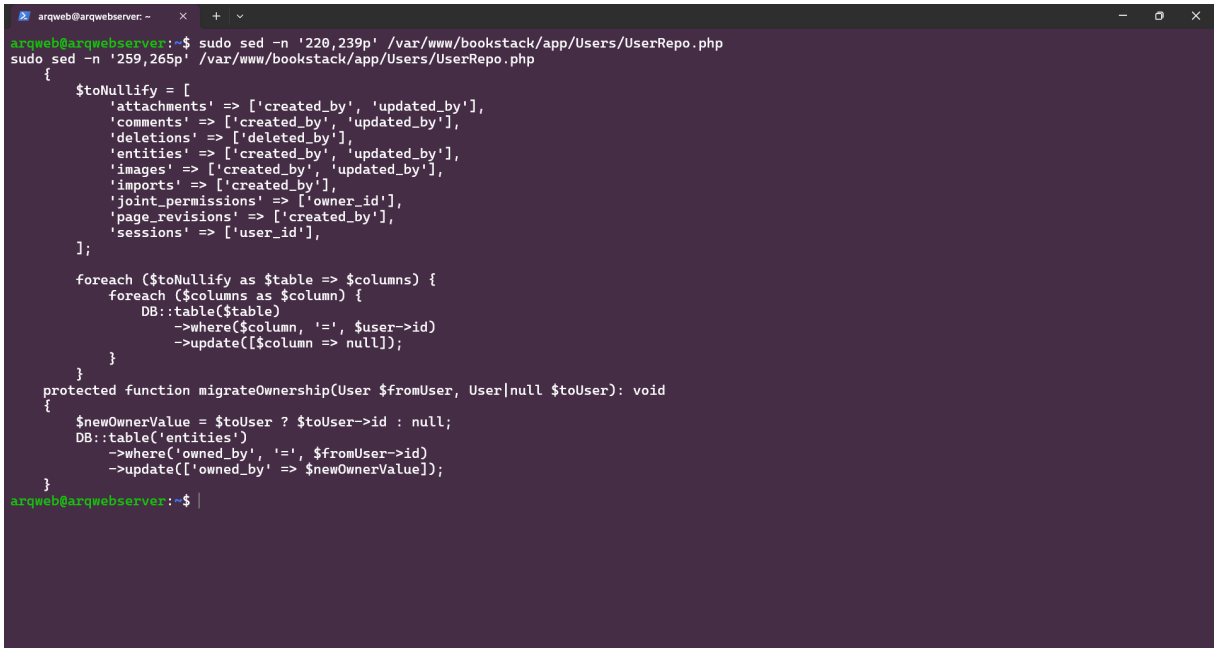


Figura 46: Respaldo de F5-E2. Actualización de autoría y propiedad sobre tablas compartidas al eliminar una cuenta.

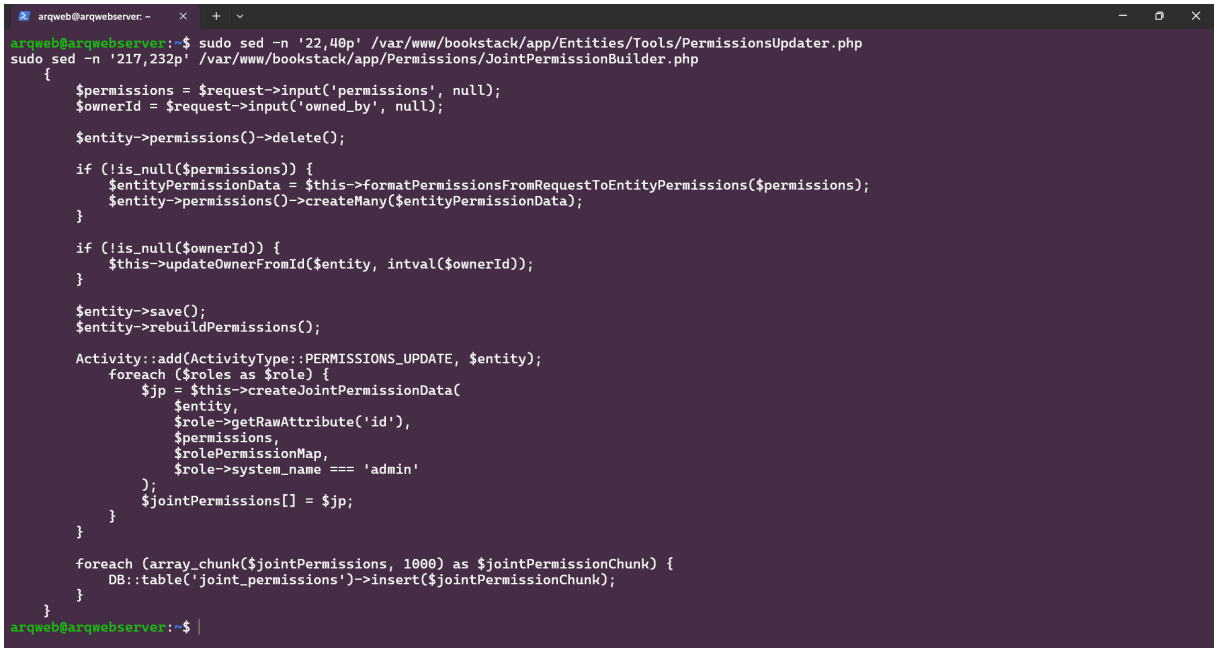


Figura 47: Respaldo de F5-E2. Persistencia de restricciones y reconstrucción de permisos efectivos.

Exportación. Los controladores de API aplican el permiso `ContentExport` y consultas visibles. El alcance de entidades une los datos específicos; `BookContents` recupera el árbol de capítulos y páginas. `ExportFormatter` obtiene Markdown o genera HTML para PDF (R06, Figuras 48 y 49). La ruta ZIP lee etiquetas y adjuntos y recupera imágenes y archivos por sus modelos (Figura 50). La extracción debe conservar tanto la visibilidad del contenido como el permiso de exportación.

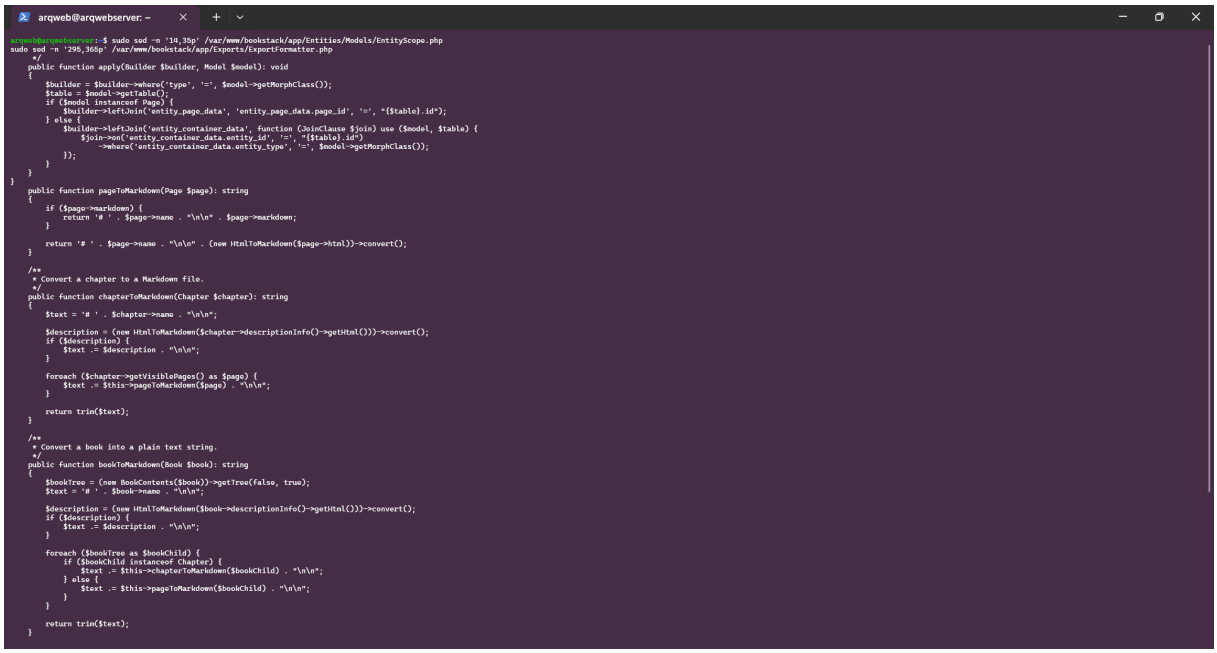


Figura 48: Respaldo de F5-E2. Alcance de entidades y conversión del contenido de página a Markdown.

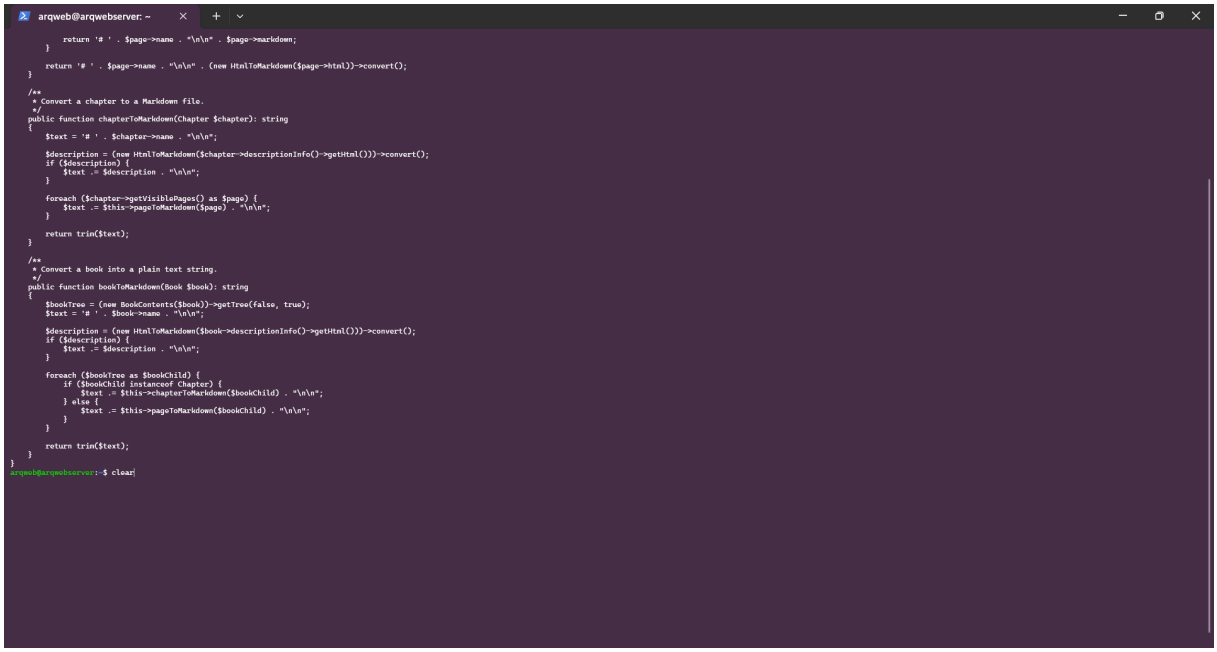


Figura 49: Respaldo de F5-E2. Recorrido de capítulos y páginas del libro durante la exportación Markdown.

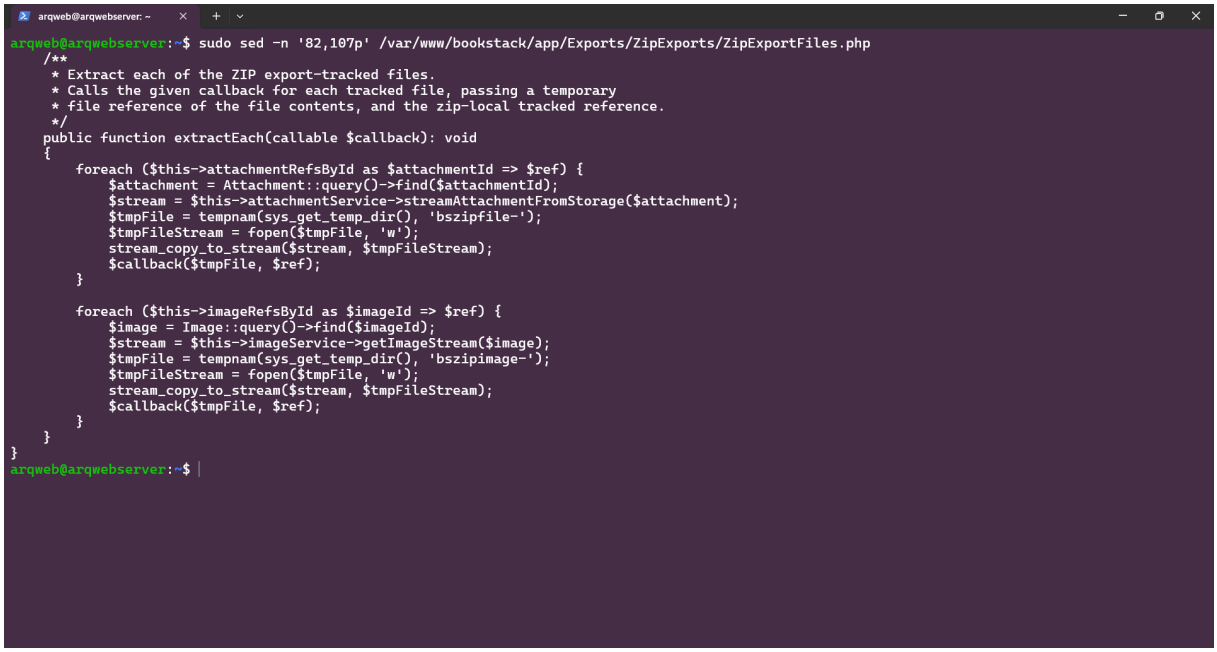


Figura 50: Respaldo de F5-E2. Lectura de adjuntos e imágenes para incorporar sus archivos a una exportación ZIP.

10.1.5. Efecto de los acoplamientos sobre la extracción

1. **Contenido y búsqueda.** El guardado de contenido llama al indexador local. La separación requiere un índice propio actualizado por eventos y lecturas de API, con manejo de actualizaciones y borrados.

2. **Contenido y autorización.** La visibilidad depende de borradores, propietario, roles y permisos efectivos. La consulta de datos mediante la API y los resultados del nuevo servicio deben conservar ese alcance.

3. **Actividad y actores.** La auditoría registra acciones del contenido y de la administración. La integración debe conservar actor, objeto, tipo y momento de la acción mediante eventos normalizados.

4. **Medios y referencias.** La limpieza considera HTML y revisiones. La gestión externa de archivos necesita preservar enlaces y referencias para evitar eliminar recursos todavía utilizados.

5. **Exportación y contenido.** La exportación recorre entidades visibles y puede incluir archivos. El servicio extraído consumirá la API con autorización y administrará sus trabajos y artefactos propios.

6. **Usuarios y propiedad.** La eliminación de una cuenta actualiza varias tablas. La representación de actores eliminados y la conservación de sus referencias deben formar parte de los contratos de integración.

10.1.6. F5-E3: Mapa de contextos

Se establecen seis contextos lógicos para analizar responsabilidades y relaciones del dominio. La Figura 51 representa su situación actual dentro de BookStack. Las flechas indican información o capacidad suministrada desde un proveedor hacia un consumidor. Los filtros de autorización se aplican transversalmente en las consultas visibles.

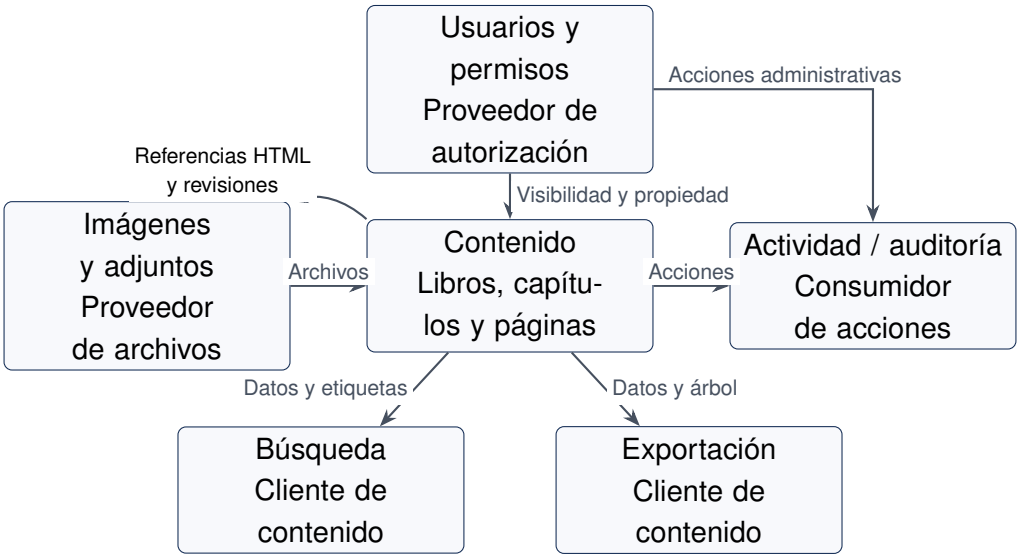


Figura 51: F5-E3. Contextos lógicos del monolito y relaciones verificadas. Los seis contextos comparten el esquema de MariaDB; la matriz detalla sus accesos.

La relación de contenido con búsqueda y exportación se caracteriza como *cliente/proveedor*. Usuarios/permisos provee las reglas de acceso; medios provee archivos y consulta referencias de contenido para su limpieza. Actividad consume las acciones producidas por contenido y administración. La persistencia compartida y las llamadas síncronas explican los límites que deben adaptarse durante la extracción.

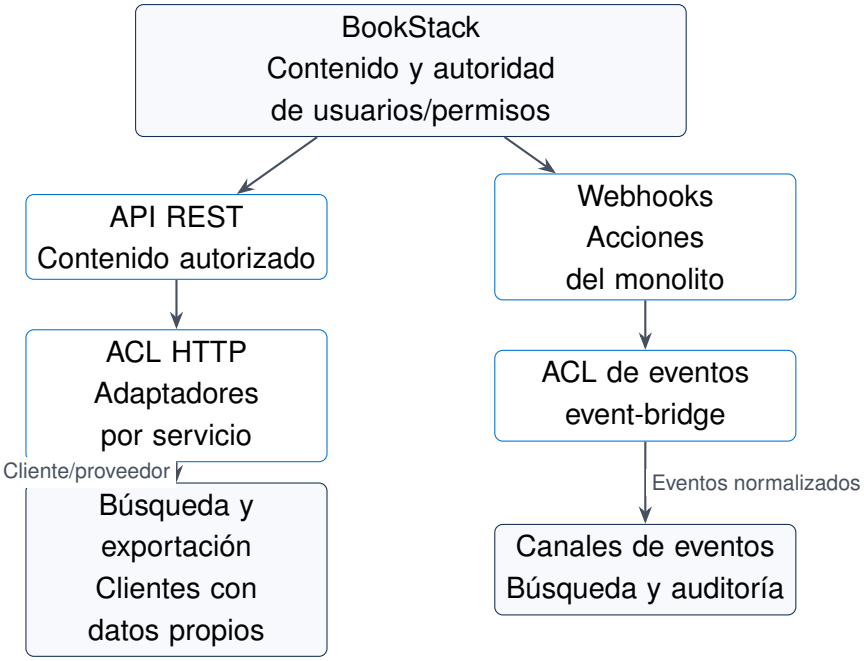


Figura 52: F5-E3. Límites de integración para la extracción. Las capas anticorrupción (ACL) adaptan la API y los Webhooks a contratos de los servicios consumidores.

La ACL HTTP traduce las respuestas del monolito a las representaciones propias de búsqueda y exportación. La ACL de eventos normaliza los Webhooks antes de entregarlos a los consumidores. Medios constituye un contexto de archivos y miniaturas con almacenamiento propio; su integración utiliza URLs y API para conservar los recursos referenciados. El diseño de contratos precisará el alcance de autorización y el ciclo de vida de los archivos. La selección de herramientas y las decisiones de consistencia se justifican en los ADR de A2.

Los fuentes editables del mapa se conservan en este archivo TikZ y en `cna/docs/fase5-a1-contextos.mmd` y `cna/docs/fase5-a1-contextos-propuesto.mmd`.

10.1.7. Matriz de pruebas de A1 y resultado

Prueba	Evidencia	Resultado verificable	Estado
P26	F5-E1	Catálogo de 41 tablas y 280 columnas; cuatro FK; asignación de persistencia a los seis módulos. Tablas 7 y 6.	Completado
P27	F5-E2	Matriz de 6 módulos por 17 tablas principales, con accesos y dependencias delegadas respaldados por código. Tablas 10–12.	Completado
P28	F5-E3	Mapa de contextos, relaciones cliente/proveedor y capas anticorrupción. Figuras 51 y 52.	Completado

Tabla 13: Continuación de la matriz consolidada para A1, desde P26.

A1 establece el inventario de los seis módulos, su acoplamiento sobre tablas compartidas y los límites de contexto para la extracción. La matriz vincula las lecturas y escrituras con el código instalado y el mapa define las relaciones funcionales y de integración que servirán de base a los ADR.

10.2. A2: Registro de Decisiones de Arquitectura

Se documentan las decisiones ADR-006 a ADR-010 aceptadas por el equipo para el diseño. Cada registro conserva contexto, decisión, alternativas descartadas y consecuencias. Las verificaciones operativas se ejecutarán durante la Fase 6. El respaldo de recursos del anfitrión se conserva en `cna/docs/adr/recursos-a2.txt`.

10.2.1. ADR-006 — Estrategia de migración e integración con el monolito

Estatus de ADR-006

Aceptado para el diseño por el equipo.

Contexto

BookStack debe continuar atendiendo la gestión documental durante la extracción de búsqueda, auditoría, exportación y medios. La consigna establece integración por API REST y Webhooks y exige conservar el código fuente del monolito.

En A1 se verificó que el guardado de entidades persiste atributos comunes y específicos, invoca la indexación y registra actividad. La visibilidad de contenido y archivos depende de usuarios y permisos efectivos. Estos acoplamientos hacen necesario definir una frontera de integración que mantenga la autorización y permita incorporar los servicios gradualmente.

Decisión

Aplicar Strangler Fig incorporando los servicios alrededor de BookStack. El Nginx de VM-1 expondrá las rutas `/v2/` de los servicios mediante el dominio `api.bookstack.local`. BookStack mantendrá la gestión documental y su dominio actual.

Los servicios que necesitan contenido lo obtendrán mediante la API REST, con una cuenta de servicio y los permisos necesarios para cada operación. Los cambios de páginas se recibirán por Webhooks en `event-bridge`, que adaptará los mensajes a los contratos de eventos y los publicará para sus consumidores. Cada adaptador HTTP o de eventos funcionará como capa anticorrupción entre los modelos del monolito y los contratos de los servicios.

La habilitación de las nuevas funcionalidades será gradual. El gateway permitirá desactivar una ruta del servicio y devolver el acceso a la funcionalidad equivalente del monolito conforme al procedimiento de reversión que se documentará en A5.

Alternativas descartadas

- **Reemplazar BookStack en una sola entrega.** Concentra el riesgo de migración de contenido, usuarios y permisos en una única intervención y dificulta conservar la operación durante la transición.
- **Integrar los servicios mediante consultas directas a MariaDB.** Acopla los servicios a las tablas compartidas observadas en A1 y puede omitir los filtros de au-

torización de BookStack. Además, incumple la restricción docente de acceso a MariaDB desde los microservicios.

- **Modificar BookStack para invocar los nuevos servicios.** Obliga a mantener cambios propios sobre el código del monolito y contradice la vía de integración exigida por la consigna.

Consecuencias

Favorables

- Permite incorporar y verificar una funcionalidad por vez manteniendo la gestión documental disponible.
- Usa las interfaces del monolito como frontera de integración y conserva sus controles de acceso en las lecturas de API.
- Facilita desactivar una ruta externa ante fallos del servicio extraído.
- Los contratos propios permiten evolucionar los servicios con persistencia independiente.

Costos y limitaciones

- La API de BookStack pasa a ser una dependencia de los servicios de búsqueda y exportación, con latencia adicional y necesidad de timeouts y tratamiento de errores.
- El índice y la auditoría externa se actualizarán de forma asíncrona, por lo que habrá un intervalo de consistencia eventual.
- La integración necesita operación adicional de gateway, adaptadores, broker y credenciales, además de observabilidad entre componentes.
- La lectura con un token de servicio exige definir el alcance de visibilidad de los resultados del servicio, especialmente en búsqueda y exportación.
- La recolección inicial y la recuperación ante eventos fallidos requieren procedimientos explícitos. Su entrega, deduplicación y reintento se especificarán en ADR-007 y ADR-010.

Respaldo y validación prevista

El inventario, la matriz y los mapas de A1 respaldan el contexto de esta decisión. En Fase 6 se verificará la lectura mediante API, el recorrido edición → Webhook → evento → reindexación y el acceso mediante gateway. La implementación se contrastará con esos resultados antes de considerar validado el diseño.

10.2.2. ADR-007 — Selección del broker de mensajes

Estatus de ADR-007

Aceptado para el diseño por el equipo. Validación operativa prevista en Fase 6.

Contexto

Una edición de BookStack debe actualizar búsqueda y auditoría mediante el mismo evento. Los consumidores necesitan posiciones independientes y recuperación de mensajes publicados mientras están detenidos. Se requieren persistencia, confirmación manual, reintentos limitados y mensajes fallidos conservados para diagnóstico. El laboratorio tiene recursos de memoria acotados.

Decisión

Usar NATS con JetStream habilitado como broker de eventos. Configurar un stream de páginas con almacenamiento en archivo sobre un volumen persistente y consumidores pull durables independientes para búsqueda y auditoría. Cada consumidor tendrá confirmación explícita después de persistir su resultado. Los trabajos de exportación conservarán su mecanismo de cola propio conforme a ADR-009.

El refinamiento del contrato A3 establece que `event-bridge` guardará inbox y outbox de forma atómica antes de devolver 202 al Webhook. El publicador esperará la confirmación de JetStream antes de marcar la salida como enviada. Mantendrá el mismo `event_id` al reintentar el evento normalizado y deduplicará la entrada conforme al contrato del puente.

El diseño inicial utiliza un broker con una réplica de almacenamiento. Esto cubre reinicios del proceso y del nodo con el mismo disco disponible. La réplica única concentra la disponibilidad del broker en ese nodo y no protege frente a pérdida del volumen.

Se usarán límites explícitos de memoria, disco y retención. La capacidad y el período de conservación deben cubrir las pruebas de caída de consumidores y medirse antes del despliegue final. La recuperación a DLQ se implementará explícitamente conforme a ADR-010. Alcanzar `MaxDeliver` en JetStream no equivale a mover automáticamente el mensaje a una DLQ.

Alternativas descartadas

Alternativa	Qué ofrece	Motivo de descarte para este diseño
RabbitMQ	Acknowledgements, publisher confirms, enrutamiento y dead-lettering configurable.	Es válido para el TP, pero requeriría adoptar su modelo de exchanges, colas y políticas. Se prefiere el modelo de stream con consumidores independientes para los eventos de páginas. El consumo relativo de RAM entre productos no se considera medido.
Redis Streams como broker general	Streams, grupos de consumidores, pendientes y recuperación de trabajo.	Redis se reserva para los trabajos de exportación. Separar el broker de eventos permite administrar su retención y consumidores sin compartir el ciclo de vida de los jobs. Redis Streams también necesitaría políticas explícitas de persistencia y mensajes fallidos.
Core NATS sin JetStream	Pub/sub entre procesos conectados.	La persistencia y recuperación de eventos durante la caída de consumidores requieren JetStream.

Tabla 14: Comparación y alcance de la decisión ADR-007.

Consecuencias

Favorables

- Un mismo evento de página puede ser procesado por búsqueda y auditoría con cursores separados.
- El consumidor puede recuperar mensajes pendientes después de un reinicio.
- Los ack explícitos permiten confirmar después de guardar el efecto del evento.

- Stream y consumidores forman una frontera de integración independiente de las tablas de BookStack.

Costos y limitaciones

- Hay que operar el volumen, los límites de retención, el monitoreo de pendientes y el procedimiento de DLQ.
- La entrega al menos una vez permite duplicados, por lo que exige idempotencia en cada servicio.
- Una instancia inicial es un punto único de falla. La indisponibilidad del broker necesita reintentos del productor y visibilidad de los fallos de Webhooks.
- Una confirmación del broker no demuestra que los consumidores ya hayan procesado el evento.
- La elección por modelo operativo debe contrastarse con consumo real. No se atribuyen cifras de ahorro de RAM sin una medición.

Respaldo y validación prevista

Guardar configuración de streams y consumidores, prueba de publicación confirmada, reinicio del consumidor y prueba de DLQ. Medir memoria y disco con la carga de laboratorio. La aceptación de Webhooks durante una caída del broker debe probarse desde BookStack; si no ofrece reintentos suficientes, habrá que agregar almacenamiento durable de entrada en `event-bridge` antes de reconocer el evento.

Fuentes

- JetStream y su modelo de persistencia y consumidores
- Consumidores pull y confirmaciones
- RabbitMQ, acknowledgements y publisher confirms
- Redis Streams

10.2.3. ADR-008 — Plataforma de orquestación

Estatus de ADR-008

Aceptado para el diseño por el equipo. La habilitación de VM-2 está condicionada a verificar memoria disponible y consumo conjunto del laboratorio.

Contexto

El TP requiere despliegue reproducible, recuperación de servicios y escalado de workers, con el monolito operativo ante la caída de servicios o VM-2. El equipo utiliza VMware y tiene una VM base con 2 vCPU y 4 GB asignados. Ubuntu reportó 3,3 GiB de RAM visible y 2,6 GiB disponibles en el relevamiento inicial.

El anfitrión Windows reportó 16.442.781.696 bytes de RAM física, 12 procesadores lógicos y 16.057.404 KiB de memoria visible. En la consulta de A2 tenía 2.249.404 KiB libres, aproximadamente 2,1 GiB. El total físico y la memoria visible miden cosas distintas y los valores libres corresponden a un instante. El respaldo se guarda en `recursos-a2.txt`.

La memoria libre observada requiere liberar aplicaciones o cargas antes de agregar una VM de 4 GB. Los recursos del anfitrión y de las VM deben verificarse de nuevo antes de instalar la plataforma y antes de las pruebas de carga.

Decisión

Adoptar Docker Swarm como orquestador del diseño, con dos VM en el laboratorio inicial.

Nodo	Rol previsto	Asignación y criterio
VM-1, `arqwebserver`	Monolito, gateway y manager de Swarm	Conservar 2 vCPU y 4 GB de la VM base. Excluir las tareas de los nuevos servicios mediante disponibilidad `drain`, conservando la función de manager.
VM-2, `cna-node1`	Worker de Swarm, servicios y persistencia	Prever 2 vCPU y 4 GB, con límites por contenedor. Confirmar que el anfitrión sostenga ambas VM antes de crearla.

Tabla 15: Comparación y alcance de la decisión ADR-008.

El manager mantendrá el estado deseado y las réplicas de los servicios. Los export-workers podrán escalar de una a tres réplicas. Las imágenes se construirán

y estarán disponibles antes de `dockerstackdeploy`; el stack no realiza la construcción de Dockerfiles.

Los almacenes con volúmenes locales se fijarán a VM-2 mediante restricciones de ubicación. Una caída de VM-2 detendrá esos componentes hasta recuperar el nodo y sus discos, mientras BookStack continuará en VM-1. El regreso del nodo debe permitir reconciliar los servicios automáticamente.

Los endpoints de servicio se publicarán de forma restringida sobre el nodo de servicios, sin abrir el routing mesh sobre todas las interfaces externas. UFW y las reglas efectivas de Docker se verificarán juntos. El tráfico entre nodos usará la red interna y los puertos de clúster necesarios, limitados por origen. El acceso externo de aplicación seguirá el gateway de VM-1.

Alternativas descartadas

- **Docker Compose sobre VM-2.** Es una base válida y de menor complejidad para un solo host. Se elige Swarm por su estado deseado, reemplazo de tareas y manejo de réplicas, útiles para los experimentos de caída y escalado. Compose queda como alternativa si las mediciones obligan a revisar este ADR, con su impacto en las pruebas y rúbrica documentado.
- **k3s.** Ofrece el modelo Kubernetes, pero agrega aprendizaje y operación de su plano de control. Sus mínimos oficiales excluyen la carga de aplicaciones. En el presupuesto inicial se prioriza una plataforma más acotada al objetivo del TP.
- **Tres VM desde el inicio.** Con la memoria libre observada, una tercera VM necesita primero demostrar capacidad del anfitrión. La observabilidad se presupuestará inicialmente en VM-2 y se trasladará solo si las mediciones justifican otro nodo.

Consecuencias

Favorables

- Permite declarar réplicas y mantener el estado deseado de servicios.
- Proporciona un mecanismo de despliegue por stack y acceso a secretos por servicio.
- Separa el ciclo de vida de servicios y monolito en dos VM.

Costos y limitaciones

- Un solo manager es un punto único de falla del plano de control. Dos nodos con un manager no constituyen alta disponibilidad.
- Los volúmenes locales fijan los servicios con estado a VM-2 y requieren respaldo y recuperación del mismo almacenamiento.

- El uso de Swarm añade puertos internos, restricciones de ubicación y reglas de publicación que deben verificarse.
- La planificación de 4 GB por VM no garantiza que todos los servicios y observabilidad entren. Los límites deben sumar menos que la memoria visible del nodo, dejando margen al SO y Docker; CPU, swap, disco y latencia se medirán bajo carga.
- El snapshot de memoria del anfitrión exige preparar el entorno antes de ejecutar dos VM simultáneamente.

Respaldo y validación prevista

Conservar mediciones de Windows y Ubuntu, listado de nodos, límites de contenedores y consumo en reposo/carga. Probar reemplazo de una tarea, escalado $1 \rightarrow 3$ de workers y recuperación tras apagar/encender VM-2. Medir BookStack mientras se ejecutan los servicios. Si no hay margen, revisar este ADR y el presupuesto antes del despliegue final.

Fuentes

- Docker Swarm, managers y workers
- Servicios, réplicas, ubicación y publicación de puertos
- Docker Compose en producción
- Requisitos de k3s

10.2.4. ADR-009 — Base de datos por servicio

Estatus de ADR-009

Aceptado para el diseño por el equipo. Versiones de imágenes, cuotas y permisos se verificarán durante la implementación.

Contexto

A1 mostró tablas compartidas de contenido, índice, actividad y autorización. Los nuevos servicios necesitan persistencia propia, reemplazo de contenedores sin pérdida de datos e independencia respecto del esquema de BookStack. MariaDB debe permanecer accesible localmente en VM-1 y los microservicios deben integrarse por API y eventos.

Decisión

Usar persistencia lógica independiente por servicio, con credenciales y permisos específicos, migraciones propias y almacenamiento fuera de la capa escribible del contenedor.

Servicio	Almacén propuesto	Propiedad de datos
S1 búsqueda	PostgreSQL, base `search_db`, rol propio	Proyección del contenido, índice de texto completo y eventos aplicados. El índice se reconstruye por API y eventos.
S2 auditoría	PostgreSQL, base `audit_db`, rol propio	Registros de eventos, actor, objeto, fecha y `request_id`; unicidad de `event_id`.
S3 exportación	Redis para trabajos; bucket `exports` en MinIO	Estados y cola de jobs, resultados y archivos con URLs prefirmadas temporales. API y workers de S3 comparten esos datos porque pertenecen al mismo servicio.
S4 medios	Bucket `media` en MinIO, credencial y política propias	Originales y miniaturas identificados por claves de objetos y metadatos.
event-bridge	PostgreSQL, base `bridge_db`, rol propio	Inbox, deduplicación y outbox de publicación definidos al refinar A3.

Tabla 16: Comparación y alcance de la decisión ADR-009.

Inicialmente `search_db` y `audit_db` podrán residir en una sola instancia PostgreSQL para limitar procesos en la VM. Cada rol tendrá `CONNECT` y permisos únicamente sobre su base. Se revocarán privilegios públicos que permitan acceso cruzado y se probará el rechazo entre roles. Ningún servicio dispondrá de un usuario superadministrador ni consultará las tablas del otro.

Los buckets compartirán una instancia MinIO con políticas separadas. Redis se dedicará a S3 y se configurará con persistencia AOF y volumen. Su política de sincronización y la recuperación de jobs se probarán antes de aceptar la durabilidad del flujo. La URL prefirmada deberá ser accesible desde el cliente a través del gateway y mantener la firma válida.

La clave y la contraseña de MariaDB no se entregarán a los servicios. La API de BookStack proveerá el contenido autorizado que necesiten. Los volúmenes persistentes se ubicarán y respaldarán conforme al ADR-008.

Alternativas descartadas

- **Compartir tablas o credenciales de MariaDB.** Conserva el acoplamiento analizado y evita la frontera de autorización y contratos establecida para los servicios.
- **Una única base con tablas y usuario compartidos para todos los servicios.** Permite consultas y cambios cruzados que comprometen la autonomía y la propiedad de datos.
- **Una instancia PostgreSQL por servicio desde el inicio.** Refuerza el aislamiento físico, pero agrega procesos y consumo fijo. Se empieza con bases y roles aislados en una instancia y se conserva la posibilidad de separarlos físicamente.
- **Meilisearch para S1.** Es una alternativa especializada válida. Se propone PostgreSQL FTS para combinar proyección, índice y deduplicación en transacciones del servicio y reutilizar operación de un solo motor relacional. La latencia de búsqueda se medirá; la elección no implica superioridad de rendimiento.
- **Archivos dentro del contenedor.** La recreación del contenedor comprometería su conservación. Se usan volúmenes del almacén y almacenamiento de objetos.

Consecuencias

Favorables

- Cada servicio evoluciona su esquema y solo expone datos mediante contratos.
- El índice y la auditoría pueden registrar el procesamiento del evento junto con su resultado en su propia transacción.
- La recreación de APIs y workers conserva los datos administrados por los almacenes.

Costos y limitaciones

- La instancia PostgreSQL compartida conserva un dominio de falla y competencia de recursos comunes, aunque las bases estén aisladas lógicamente.
- MinIO compartido también concentra disponibilidad y disco. Las políticas deben impedir acceso de S4 al bucket de S3 y viceversa.
- Redis, PostgreSQL y MinIO requieren volúmenes, cuotas, backup y pruebas de restauración.
- Los datos replicados por eventos tienen consistencia eventual y necesitan procedimientos de recuperación.
- AOF, volúmenes y confirmaciones no sustituyen la prueba de durabilidad frente al tipo de falla elegido.

Respaldo y validación prevista

Guardar el inventario de almacenes, permisos por rol/bucket, pruebas negativas de acceso cruzado, recreación de contenedores y recuperación de datos. Verificar en VM-1 que ningún servicio abra conexiones a MariaDB. Registrar la persistencia de trabajos de exportación y la descarga de archivos por el gateway.

Fuentes

- Privilegios de PostgreSQL
- Persistencia de Redis

10.2.5. ADR-010 — Consistencia eventual, idempotencia y mensajes fallidos

Estatus de ADR-010

Aceptado para el diseño por el equipo. Las garantías se verificarán con pruebas de duplicación, reentrega, caída y recuperación.

Contexto

La búsqueda y la auditoría se actualizan de forma asíncrona. Un mensaje puede entregarse nuevamente si el consumidor guarda el resultado y falla antes de confirmar. También puede fallar de forma transitoria o permanente, llegar fuera de orden o agotar su ventana de conservación. La edición de una página debe reflejarse en el índice en menos de cinco segundos según los criterios del TP.

Decisión

Adoptar consistencia eventual y entrega al menos una vez, con efectos idempotentes por consumidor. El identificador `event_id` pertenece al evento lógico y se mantiene en sus reintentos y reenvíos a DLQ. El sobre incluye además `type`, `version`, `occurred_at`, `request_id` y `payload`, que se definirán en AsyncAPI durante A3.

Persistencia y confirmación

1. Validar el sobre y la versión antes de aplicar el evento.
2. En auditoría, usar una restricción única por `event_id`. La inserción y cualquier marca de procesamiento se confirman en una misma transacción local.
3. En búsqueda, mantener el registro del evento aplicado y la actualización de la proyección en una misma transacción de `search_db`. La consulta a la API ocurre con timeout y tratamiento de errores; el guardado se serializa por entidad y respeta versiones de contenido cuando estén disponibles.
4. Confirmar el mensaje al broker después de completar la transacción. Si el evento ya está aplicado, reconocerlo sin repetir el efecto.
5. Ante una falla previa al commit, permitir la reentrega. Si el commit se completó pero se perdió el ack, la restricción de idempotencia evita el efecto duplicado.

El registro de deduplicación debe conservarse como mínimo durante la retención y la ventana de replay de los eventos. Cada consumidor tiene su registro propio, por lo que S1 y S2 pueden procesar legítimamente el mismo `event_id` una vez cada uno.

Orden y reconciliación

S1 consultará el estado actual de la página y serializará sus actualizaciones por entidad. Se documentará el criterio de versión o reconciliación para evitar que una respuesta antigua sobrescriba una nueva. Un borrado conserva una marca de eliminación o un criterio equivalente para evitar resurrección por eventos atrasados. Un error de autorización o un timeout no se interpretará como prueba de borrado.

La indexación inicial y la reconciliación por API complementan los Webhooks. El orden de creación del consumidor y de la carga inicial, junto con el replay, debe cubrir las ediciones concurrentes con esa carga. Las condiciones exactas se formalizarán en los contratos y pruebas de S1.

Reintentos y DLQ

Se propone un máximo de cinco intentos por consumidor, con esperas crecientes y jitter, y timeouts explícitos hacia BookStack. Los errores permanentes de esquema o versión se envían a DLQ con su causa, mientras los transitorios usan reintentos acotados.

La DLQ conservará el sobre original, el consumidor, la causa, los intentos y las marcas temporales. El traslado espera confirmación de persistencia en el destino antes de reconocer o terminar el original. Si ocurre una caída entre esos pasos, la duplicación en la DLQ se controla con la clave `(consumer,event_id)`.

JetStream requiere implementar esa transferencia. Se supervisarán también los avisos de entrega agotada para recuperar mensajes que lleguen al límite después de la caída de un consumidor. Los mensajes agotados deben quedar recuperables en el stream hasta confirmar su traslado. El monitoreo alertará por DLQ y por mensajes agotados pendientes de transferencia.

El replay conserva el `event_id`. Un evento que nunca se aplicó podrá procesarse tras corregir su causa. Una reparación que busque cambiar un efecto ya aplicado requerirá una operación explícita de reconciliación o compensación, con un nuevo identificador y relación con el original.

Trabajos de exportación

La idempotencia de una solicitud HTTP se tratará mediante una clave de solicitud y su `job_id`, separada del `event_id` de sus eventos. La creación del estado y la inserción en la cola de Redis deben ser atómicas. El worker tendrá un mecanismo de reserva recuperable y actualizará estado y resultado de forma idempotente. Una caída no debe dejar un job en `running` indefinidamente. El objeto tendrá una clave estable por job para permitir reintentos sin multiplicar resultados.

Alternativas descartadas

- **Confirmar antes de guardar.** Una caída entre ack y persistencia puede perder el efecto de un mensaje ya reconocido.
- **Suponer entrega exactamente una vez para toda la operación.** La transacción del almacén y el ack del broker son pasos diferentes. Las capacidades de deduplicación del broker no sustituyen la idempotencia del efecto de negocio.
- **Reintentar indefinidamente.** Un mensaje inválido puede consumir capacidad y ocultar una falla permanente.
- **Deduplicar solo en memoria.** El reinicio del proceso pierde el registro de eventos aplicados.

- **Descartar al agotar intentos.** Impide diagnosticar y recuperar el evento, contrario al objetivo de entrega confiable del TP.

Consecuencias

Favorables

- Permite recuperar eventos tras una caída y evitar efectos duplicados verificables.
- Los fallos permanentes quedan disponibles para diagnóstico y replay controlado.
- Las restricciones locales y el ack posterior mantienen consistente la proyección de cada servicio frente a reentregas.

Costos y limitaciones

- El usuario puede observar temporalmente datos anteriores en búsqueda mientras se procesa un evento.
- Se requieren tablas de eventos procesados, versiones o serialización por entidad, cuotas y limpieza compatible con el replay.
- La DLQ y su transferencia agregan código y operación y necesitan pruebas de caída en sus puntos intermedios.
- El umbral de cinco segundos depende de carga, disponibilidad y reintentos. Su cumplimiento se medirá desde la edición hasta la consulta del índice.
- La durabilidad comienza en la aceptación persistente de cada tramo. Se deberá comprobar también la recuperación del tramo BookStack $\rightarrow$ event-bridge.

Respaldo y validación prevista

Enviar el mismo evento dos veces y verificar un solo efecto por consumidor. Detener auditoría mientras se editan 20 páginas y comprobar su recuperación. Forzar un error permanente y observar la DLQ. Interrumpir el consumidor después del commit y antes del ack. Probar orden invertido de actualización/borrado y caída de un worker durante un job. Conservar logs, IDs y tiempos de cada experimento.

Fuentes

- Persistencia y entrega de JetStream
- Consumidores y confirmaciones
- Redis Streams y grupos de consumidores

10.2.6. Resultado de A2

Registro	Decisión aceptada para el diseño
ADR-006	Strangler Fig mediante API REST y Webhooks.
ADR-007	NATS JetStream con consumidores durables y confirmación explícita.
ADR-008	Swarm en dos VM, con verificación de memoria previa a habilitar VM-2.
ADR-009	Persistencia por servicio, aislamiento de bases, roles y buckets.
ADR-010	Consistencia eventual, efectos idempotentes, reintentos y DLQ.

Tabla 17: Cierre de A2 y decisiones de arquitectura.

Los cinco ADR conservan alternativas descartadas y consecuencias negativas. Su aceptación cierra el producto de diseño A2 y establece las decisiones que se verificarán durante la construcción y las pruebas de las siguientes fases.

10.3. A3: Contratos de API y de Eventos

Se definieron los contratos HTTP de los cuatro servicios de dominio y del adaptador de Webhooks, junto con el catálogo de eventos de integración. Los documentos se conservan en `cna/contracts/`. Las especificaciones fijan las entradas, salidas, errores, correlación y compatibilidad que implementarán los servicios de la Fase 6.

10.3.1. Contratos HTTP en OpenAPI 3.1

Los cinco documentos utilizan `openapi: 3.1.0`, esquemas de datos, parámetros, ejemplos, seguridad y respuestas tipadas. La Tabla 18 resume las rutas de negocio. Los cuatro servicios de dominio usan el servidor externo `https://api.bookstack.local`. El adaptador opera exclusivamente en la red privada.

Documento	Operaciones principales	Resultado
`search-svc.openapi.yaml`	GET `/v2/search?q=`	Libros y páginas con puntaje, resaltado y enlace al monolito.
`audit-svc.openapi.yaml`	GET `/v2/audit`; GET por `event_id`	Actividad filtrada por entidad, ID y rango temporal; consulta individual de evento.
`export-svc.openapi.yaml`	POST `/v2/exports`; GET por `job_id`	Aceptación 202, consulta de estados y archivo con URL prefirmada temporal.
`media-svc.openapi.yaml`	POST `/v2/media`; GET de metadatos, original y miniatura	Imagen persistida, miniatura proporcional y evento de subida.
`event-bridge.openapi.yaml`	POST `/internal/webhooks/bookstack`	Entrada autenticada, deduplicación y aceptación durable del Webhook.

Tabla 18: Contratos HTTP de A3 y rutas principales.

Cada servicio expone además `/healthz`, `/readyz` y `/metrics` mediante servidores internos por operación. La sonda de vida verifica el proceso; la de disponibilidad comprueba las dependencias necesarias para aceptar trabajo. Las métricas incluyen solicitudes por código y un histograma de duración en formato Prometheus.

Las respuestas **200** entregan resultados disponibles y **202** aceptan operaciones asíncronas. **400** identifica una entrada inválida, **404** un recurso inexistente o fuera del alcance visible, y **503** una indisponibilidad. Una búsqueda sin coincidencias devuelve 200 con una lista vacía. Los errores comparten el esquema `Error`, con código, mensaje y `request_id`. Se documentan también 401/403 para autorización, 409 para conflicto de idempotencia, 413/415 para subidas y 429 para rate limiting.

La exportación admite exactamente un origen, página o libro, y formato PDF o Markdown. El trabajo recorre `queued`, `running`, `done` o `failed`. El esquema de `done`

exige el archivo y su vencimiento; `failed` exige una causa. La clave `Idempotency-Key` permite reconocer una solicitud repetida y rechaza su reutilización con un contenido distinto.

Medios acepta PNG, JPEG y WebP, con límite inicial de 10 MiB y 20 millones de píxeles. La miniatura conserva proporción y se limita a 320 por 320 píxeles. Estas restricciones forman parte del contrato de esta versión.

Las rutas externas requieren una credencial de consumidor `X-API-Key`, administrada como secreto y separada del token interno de BookStack. El servicio debe validar el alcance de acceso antes de entregar texto, auditoría o archivos. La implementación inicial del laboratorio se limita al contenido compartido autorizado para la cuenta de servicio y sus consumidores. La autorización diferenciada por usuario requiere proyectar o consultar sus permisos antes de ampliar ese alcance.

10.3.2. AsyncAPI 3.0 y sobre de evento común

El archivo `bookstack.asyncapi.yaml` utiliza `asyncapi: 3.0.0`. Cada canal define su mensaje, el esquema del payload, un ejemplo y las operaciones de envío y recepción identificadas por componente. Los consumidores de páginas de búsqueda y auditoría son durables e independientes, con confirmación explícita.

Canal	Productor	Consumidores
`bookstack.page.created`	event-bridge	Búsqueda y auditoría
`bookstack.page.updated`	event-bridge	Búsqueda y auditoría
`bookstack.page.deleted`	event-bridge	Búsqueda y auditoría
`export.requested`	API/outbox de exportación	Auditoría
`export.completed`	Worker/outbox de exportación	Auditoría
`media.uploaded`	Medios	Auditoría

Tabla 19: Los seis canales de integración requeridos por A3.

El sobre común exige `event_id` de tipo UUID, `type`, `version`, `occurred_at` en ISO 8601 con zona, `request_id` y `payload`. Se incorpora `traceparent` opcional para la instrumentación de trazas. El tipo del evento es constante para cada mensaje, y los payloads definen actor, entidad y datos propios del hecho. Una exportación completada publica la clave durable del objeto; la URL prefirmada temporal se obtiene por HTTP.

El siguiente ejemplo ilustra el contrato de actualización de página.

```
{
  "event_id": "4e07d48d-e610-4b7f-9d2c-a10e960ec8b7",
  "type": "bookstack.page.updated",
  "version": "1.0",
```

```
  "occurred_at": "2026-11-11T14:00:00Z",
  "request_id": "9c7059e1b4954ed8934ba05643761378",
  "payload": {
    "page_id": 42,
    "book_id": 7,
    "actor": {"id": 3, "name": "Usuario de laboratorio"},
    "source_revision": 2
  }
}
```

Los jobs ejecutables se reservan en Redis. El evento `export.requested` describe el hecho para otros consumidores y evita duplicar la ejecución del mismo trabajo en otra cola. La persistencia del job y de su publicación pendiente debe ser atómica. La deduplicación y el tratamiento de fallos siguen ADR-010.

10.3.3. Adaptación del Webhook y aceptación durable

El formato de entrada se contrastó con `WebhookFormatter` de la instalación relevada. El puente convierte `page_create`, `page_update` y `page_delete` a los tres tipos normalizados. Obtiene el actor de `triggered_by`, el momento de `triggered_at` y el ID de página de `related_item.id`. Tolera los campos adicionales de BookStack y copia solo los definidos por su contrato.

El token compartido en query se valida antes de persistir y su valor se redacta en los logs del puente y proxy. El código relevado de `DispatchWebhookJob` registra fallos HTTP o de conexión sin programar un nuevo intento de esa llamada. Por ello se refinan ADR-007 y ADR-009 para que el puente guarde inbox y outbox en una transacción de `bridge_db` antes de responder 202. El publicador confirma JetStream antes de marcar el outbox como enviado. La caída temporal del broker permite recuperar la publicación desde esa entrada persistida.

Una repetición idéntica del Webhook conserva el UUID normalizado y recibe 200. La huella del JSON canónico de entrada se conserva durante al menos siete días. La falta de identificador único de origen impide distinguir dos acciones diferentes con cuerpos idénticos; ese límite se comprobará con los eventos y revisiones reales durante la implementación.

El gateway inicia `X-Request-ID` y los servicios lo propagan a las respuestas, llamadas, logs y eventos. El Webhook relevado no incorpora por sí mismo el identificador de la edición original. El puente genera uno cuando está ausente. La instrumentación del envío desde BookStack deberá propagar el contexto para completar la prueba de trazas de extremo a extremo. La recuperación del índice por API complementa los Webhooks; recuperar toda la auditoría de entradas que nunca llegaron al puente requiere un procedimiento específico de origen.

10.3.4. Política de versionado y compatibilidad

La versión de la ruta HTTP es independiente de la versión del esquema de evento y de la revisión del documento de contrato.

1. La API utiliza `/v2/`, con revisión documental `info.version: 1.0.0`. Un cambio incompatible crea `/v3/` y mantiene ambas rutas durante la migración, conservando el contrato de v2 para los consumidores existentes.
2. Los seis subjects actuales conservan mensajes con `version: 1.0`. Los esquemas de sobre y payload son estrictos, por lo que agregar campos requiere una evolución planificada del esquema.
3. Una versión incompatible de evento se publica en subjects separados, por ejemplo `v2.bookstack.page.updated`, con `version: 2.0`. Los consumidores antiguos mantienen sus subjects y los nuevos reciben la representación compatible con su contrato.
4. Durante la transición se pueden publicar ambas representaciones del mismo hecho, conservando el ID lógico y la correlación. Cada proyección deduplica en su propio almacén y solo consume la versión para la que fue diseñada.

10.3.5. F5-E4: Validación de especificaciones y ejemplos

La validación reproducible se ejecutó desde el directorio de contratos mediante `nodevalidate.mjs`. Las dependencias se fijan con `pnpm-lock.yaml`; su instalación se verificó mediante `pnpminstall--frozen-lockfile--ignore-scripts`. El procedimiento completo está en `cna/contracts/README.md`.

Spectral 6.17.1 valida los cinco OpenAPI; el parser oficial AsyncAPI 3.6.6 valida el catálogo de eventos. AJV 8.20.0 contrasta los ejemplos con los esquemas. La ejecución final comprobó **242 ejemplos**, **9 rechazos esperados de entradas inválidas** y **0 errores**. Las entradas negativas incluyen orígenes de exportación ambiguos, trabajos terminados sin archivo, UUID inválido, correlación ausente y versiones incompatibles.

El parser emite una recomendación informativa para adoptar AsyncAPI 3.1. Se mantiene 3.0 por la consigna docente. La ejecución registra cero errores y cero advertencias. Los resultados completos, versiones de herramientas, usuario, hostname y hashes SHA-256 se guardan en `cna/docs/evidencias/fase5/a3/validation-report.json` y `cna/docs/evidencias/fase5/a3/validation-summary.txt`.

Prueba	Evidencia	Resultado
P29	F5-E4	Cinco OpenAPI 3.1 y un AsyncAPI 3.0 validados; seis canales, sobre común y política de versionado documentados; ejemplos y rechazos comprobados. Resultado PASS.

Tabla 20: Validación de los contratos de A3 y continuación de la matriz de pruebas.

```
PS C:\Users\sofii> Set-Location 'C:\Users\sofii\tp-arqweb\cna\contracts'
PS C:\Users\sofii\tp-arqweb\cna\contracts>
PS C:\Users\sofii\tp-arqweb\cna\contracts> & 'C:\Users\sofii\.cache\codex-runtimes\codex-primary-runtime\dependencies\no
de\bin\node.exe' validate.mjs
F5-E4 / P29 — 2026-11-11T17:41:57.938Z
Usuario: sofii. Nodo: SOFI_PORTATIL. Node: v24.22.1
search-svc.openapi.yaml: PASS — Spectral, 0 diagnósticos
audit-svc.openapi.yaml: PASS — Spectral, 0 diagnósticos
export-svc.openapi.yaml: PASS — Spectral, 0 diagnósticos
media-svc.openapi.yaml: PASS — Spectral, 0 diagnósticos
event-bridge.openapi.yaml: PASS — Spectral, 0 diagnósticos
bookstack.asyncapi.yaml: PASS — AsyncAPI, 0 errores, 0 advertencias, 1 info
Ejemplos: 242. Rechazos verificados: 9. Errores: 0. Resultado: PASS
PS C:\Users\sofii\tp-arqweb\cna\contracts>
```

Figura 53: F5-E4. Ejecución de los validadores de contratos con identificación del equipo y resultado final.

Los contratos, el versionado y sus salidas de validación constituyen el producto de diseño A3. La implementación de los endpoints, la durabilidad del puente, el alcance de autorización y las interacciones se verificarán en la Fase 6 conforme a estas especificaciones.